ASSIGNMENT - 1

CNN Design & Gradient-Based Training

Assignment title	CNN Design & Gradient-Based Training
Course	CSA4701 Deep Learning
Assessment type	Assignment
Total marks	100
Student name	Y. Varun Kumar Reddy
Register number	192424230
Department	B Tech AI & DS
College	Saveetha Institute of Medical & Technical Sciences
Faculty name	VINCENT GNANARAJ T
Academic year	2024
Submission date	20-10-2026

1. Assignment Overview

1.1 What this assignment is about

This assignment asks you to design, analyse, train and evaluate a Convolutional Neural Network (CNN) that classifies images into predefined categories. You will work through the complete pipeline of a small deep-learning project: preparing an image dataset, choosing the layers of a CNN, calculating by hand the output shape and number of trainable parameters of every layer, selecting a weight-initialization scheme and a gradient-based optimizer, applying regularization to control overfitting, and finally reporting learning curves and results on unseen test data.

The official assessment sheet describes the task as follows:

> **Official assignment description (from the assessment sheet)**
> **Title:** CNN Design & Gradient-Based Training **Type:** Assignment
> **Topics covered:** Convolution, pooling, stride/padding, initialization, CNN training.
> *"Design and train a CNN for an image-classification problem. Compute output shapes and parameter counts for each layer, select suitable initialization and optimizer, apply regularization, and report learning curves and test results."*

Everything else in this document — the dataset choice, the numbered questions, the marks, the experiments and the report templates — is a **proposed addition** made to turn the one-sentence description into a complete, implementable assignment. These additions are marked as "proposed" wherever the distinction matters, so that faculty can adjust them without altering the official requirements.

1.2 Why CNNs are used for image classification

A colour image is a grid of numbers. A 32 × 32 RGB image already contains 3,072 values, and a fully connected network that connects every pixel to every hidden unit needs a very large number of parameters and ignores the fact that neighbouring pixels are related. CNNs exploit three properties of images:

- **Local connectivity.** Each output value of a convolutional layer depends only on a small neighbourhood (the kernel window) of the input, so the network learns local patterns such as edges, corners and textures.
- **Weight sharing.** The same kernel is slid across the whole image, so a pattern learnt in one position is detected everywhere. This greatly reduces the number of parameters compared with a dense layer.
- **Hierarchical features.** Stacked convolution and pooling layers combine simple features into progressively more abstract ones (edges → shapes → object parts → objects), which a final classifier maps to class probabilities.

Pooling layers then reduce the spatial size of the feature maps, which lowers computation and gives the network a small amount of tolerance to shifts of the object in the image [1].

1.3 Purpose of gradient-based training

The kernels and weights of a CNN are not designed by hand; they are learnt from labelled examples. A loss function measures how wrong the network's predictions are. Backpropagation computes the

gradient of the loss with respect to every trainable parameter, and an optimizer uses these gradients to change the parameters in the direction that reduces the loss. Repeating this over many mini-batches and epochs is what we call gradient-based training. The quality of the final model depends strongly on the starting point (initialization), the update rule (optimizer and learning rate) and the measures taken against memorizing the training data (regularization).

1.4 Expected learning outcomes

Theoretical outcomes. After completing the assignment you should be able to:

- explain how convolution, stride, padding and pooling transform an image tensor, and derive the output shape of each layer;
- derive the number of trainable parameters of convolutional and dense layers and distinguish them from hyper-parameters and non-trainable quantities;
- explain why initialization matters and justify a choice for ReLU-based networks;
- describe gradient descent, backpropagation and common optimizers, and justify an optimizer and learning rate;
- describe how dropout, weight decay, data augmentation and early stopping reduce overfitting.

Practical outcomes. You should be able to:

- implement and train a CNN in Python with a standard deep-learning framework;
- design controlled experiments that change one major factor at a time;
- produce and interpret learning curves, a confusion matrix and a classification report;
- evaluate a final model fairly on a test set that was never used for tuning, and report results honestly and reproducibly.

1.5 How the topics fit together

The topics of this assignment form one chain, shown in Table 1. Each stage depends on the previous one, and a weakness at any stage shows up in the learning curves at the end.

Table 1: *Relationship between the topics of the assignment*

Stage	Role in the pipeline	Depends on / affects
Convolution	Extracts local spatial features using learnable kernels	Kernel size, stride and padding decide the shape of every later layer
Pooling	Reduces spatial size and adds small shift tolerance	Shrinks the input to the dense layers and so controls their parameter count
Initialization	Sets the starting values of all weights	Affects whether gradients neither vanish nor explode, and so how fast training starts
Gradient-based optimization	Updates weights to reduce the loss	Uses gradients from backpropagation; learning rate and optimizer shape convergence
Regularization	Limits over-fitting to the training set	Changes the gap between training and validation curves
Evaluation	Measures generalization on unseen data	Valid only if the test set was kept separate from all tuning decisions

References in square brackets, such as [1], refer to the list in Section 17.

2. Problem Statement

2.1 Problem objective

Design and train a CNN that classifies 32 × 32 colour photographs into one of ten object categories, and document the design decisions, calculations, experiments and results. The objective is not only to obtain a high accuracy but to **justify every design choice** and to report the outcome honestly.

2.2 Dataset: CIFAR-10 (proposed default)

The default dataset is **CIFAR-10** [2], a standard benchmark of 60,000 colour images of size 32 × 32 pixels in 10 classes, with 6,000 images per class. The creators supply a split of 50,000 training images and 10,000 test images. It is proposed because:

- the images are small, so a CNN can be trained on a laptop CPU or a free GPU in a reasonable time;
- there are enough images and enough variety for overfitting to be clearly visible, which makes the effect of regularization measurable;
- the 10 classes are balanced, so accuracy is a meaningful headline metric (precision, recall and F1 are still required);
- it is built into TensorFlow/Keras and PyTorch/torchvision, and so is easy to load reproducibly;
- the 32 × 32 size gives clean shape arithmetic when 2 × 2 pooling is applied repeatedly (32 → 16 → 8 → 4).

Dataset source used for the experiments in this report

The official download site of CIFAR-10 (cs.toronto.edu) was not reachable from the computer on which the experiments were run. The images were therefore taken from a public GitHub mirror that stores the CIFAR-10 images as PNG files (repository YoongiKim/CIFAR-10-images). The mirror was checked for size and class counts: 50,000 training images (5,000 per class) and 10,000 test images (1,000 per class), all 32 × 32 RGB, in the 10 classes of Table 2. It could not be compared byte-for-byte with the official batch files, so absolute accuracies may differ slightly from runs on the official files.



Figure 1: *Random sample of eight training images from each of the ten CIFAR-10 classes*

2.3 Input format and output classes

Item	Specification
Raw input	One RGB image, 32 × 32 pixels, 3 channels, 8-bit integers in the range 0–255
Tensor shape fed to the network	(height, width, channels) = (32, 32, 3) per image (channels-last, the Keras default); a batch has shape (B, 32, 32, 3)
Pre-processing	Convert to float32 and standardise each channel using the mean and standard deviation computed on the training split only
Label format	Integer class index 0–9 (used with sparse categorical cross-entropy)
Network output	A vector of 10 class probabilities produced by a softmax layer; the predicted class is the one with the largest probability

Table 2: *CIFAR-10 class labels*

Label	0	1	2	3	4	5	6	7	8	9
Class	airplane	automobile	bird	cat	deer	dog	frog	horse	ship	truck

2.4 Training, validation and test datasets

The ten thousand official test images must be used **only once**, to report the final result. A validation set is carved out of the training images for tuning and monitoring (proposed split):

Table 3: *Proposed dataset split*

Subset	Images	Source	Used for
Training	45,000	90% of the official training images, stratified by class	Updating the weights; computing normalization statistics
Validation	5,000	10% of the official training images, stratified by class	Monitoring each epoch, early stopping, choosing hyper-parameters
Test	10,000	Official CIFAR-10 test images	One final, unbiased evaluation of the chosen model

2.5 Expected model outputs

- A trained CNN that maps a (32, 32, 3) image to 10 class probabilities.
- A saved model file and a saved training history (loss and accuracy per epoch for training and validation data).
- Learning-curve plots, a confusion matrix and a per-class classification report on the test set.
- Written calculations of output shape and trainable parameters for every layer, and a results discussion.

2.6 Performance evaluation criteria

The model is judged on **how well the work is done and explained**, not on beating a fixed accuracy. The quantities to be reported are:

- **Test accuracy** of the final model, together with test loss.
- **Precision, recall and F1-score** for each class and their macro averages, and the **confusion matrix**.
- **Training and validation accuracy and loss** after every epoch (learning curves) and the gap between them.
- **Correctness of the hand calculations** compared with the framework's own model summary.
- **Soundness of the experimental method**: controlled comparisons, a clean test set and reproducible settings.

2.7 Practical constraints and assumptions

- Training must complete on a standard laptop CPU or a free cloud GPU. Choose a model small enough to do so; the reference design in Section 5 has 356,810 trainable parameters.
- No pre-trained weights or transfer learning are allowed, since the assignment is about designing and training a CNN from scratch.
- All random seeds are fixed and reported. Small differences between runs can remain on GPUs because some operations are non-deterministic.
- All numerical results must come from your own runs. Any value copied from another source must be clearly cited.
- Biases are included in every convolutional and dense layer unless you state otherwise.

2.8 Official requirements versus proposed assumptions

Table 4: *What is official and what is proposed*

Aspect	Official (assessment sheet)	Proposed addition (adjustable by faculty)
Task	Design and train a CNN for an image-classification problem	CIFAR-10 chosen as the default problem
Calculations	Compute output shapes and parameter counts for each layer	Formulas, worked examples, a reference architecture and practice questions
Training choices	Select suitable initialization and optimizer; apply regularization	He initialization, Adam and SGD with momentum, dropout and L2 as suggested defaults
Reporting	Report learning curves and test results	Confusion matrix, classification report, result tables, test plan
Topics	Convolution, pooling, stride/padding, initialization, CNN training	Gradient-based optimization, regularization and model evaluation are treated as sub-topics of "CNN training"
Marks and duration	Not specified	50 marks with the distribution in Section 13; duration to be set by faculty

3. Assignment Objectives

The ten measurable objectives below are what you are expected to demonstrate. The right-hand column shows the evidence that shows each objective has been met.

Table 5: *Measurable objectives*

No.	Objective	Evidence of achievement	Section
1	Understand convolution and feature extraction	Correct explanation in Q1; feature maps described in the design task	4, 5
2	Calculate output dimensions for convolutional and pooling layers	Correct shapes in Q2, Q3 and the layer table, matching the framework summary	4, 5, 6
3	Calculate trainable parameters for every model layer	Correct per-layer and total counts in Q4 and the layer table; model.count_params() agrees	4, 5, 6
4	Compare suitable weight-initialization methods	Reasoned comparison of Xavier/Glorot and He in Q5; effect measured in experiments	4, 9
5	Implement a CNN for image classification	Working Python code that builds and trains the model	7
6	Select and justify a gradient-based optimizer	Justified choice in Q6; comparison of two optimizers in an experiment	4, 9
7	Apply regularization to reduce overfitting	Dropout and/or weight decay applied; effect on the train–validation gap reported	4, 9
8	Train the model and record learning curves	Saved history and accuracy and loss plots for every experiment	7, 10
9	Evaluate the final model on unseen test data	Test accuracy, confusion matrix and classification report from a test set used once	7, 12
10	Analyse the results and recommend improvements	Discussion of strengths, limitations and next steps based on measured evidence	12, 16

4. Theory and Conceptual Questions

Each question is followed by its **answer** (marked with a green bar). Calculations show the formula, the substitution and every intermediate step. Marks are shown beside each question; the mapping of questions to the marking scheme is in Section 13. The questions themselves are **proposed additions** to the official description; the answers use measured values from the experiments of Section 9 wherever a question refers to them.

Q1. CNN Architecture and Image Classification [1 mark]

A CNN for image classification is usually built from a feature-extraction part (convolution, activation and pooling layers) followed by a classification part (flatten or global average pooling, then dense layers ending in softmax).

1. Draw a block diagram of a CNN for 32 × 32 RGB images and label each block.
2. Explain what a **kernel (filter)** is and what a **feature map** is. If a layer has 32 filters, how many feature maps does it produce for one input image?
3. Explain how early convolutional layers differ from deeper ones in the kind of features they tend to detect.
4. Explain **local connectivity** and **weight sharing**, and why they make a convolutional layer cheaper than a dense layer for images.

Answer. 1. Block diagram. Figure 2 shows the network designed and trained in this report. Three convolution–pooling blocks form the feature-extraction part. Flatten, one hidden dense layer with dropout and a 10-way softmax layer form the classification part. Under every block the figure gives the output shape and the number of trainable parameters.

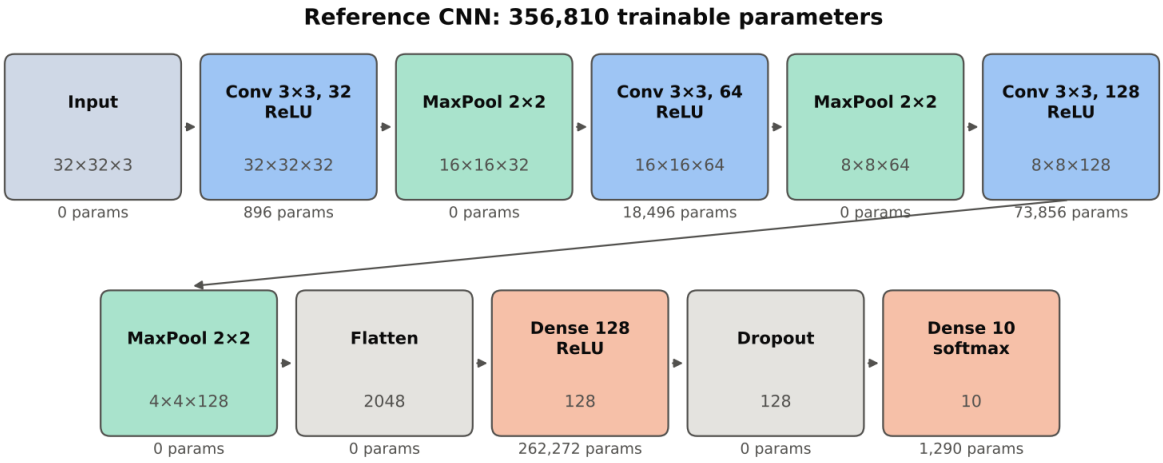


Figure 2: *Block diagram of the CNN designed for CIFAR-10 (output shapes and trainable parameters per layer)*

2. Kernel and feature map. A kernel (filter) is a small grid of learnable weights — here 3 × 3 across all input channels. It is slid over the input; at each position it computes a weighted sum of the covered values plus a bias, and ReLU is then applied. The grid of numbers produced by one kernel is a feature map: it shows where in the image the pattern that this kernel responds to is present. A layer with 32 filters therefore produces **32 feature maps** for one input image (an output of depth 32).

3. Early versus deep layers. A unit in the first convolutional layer sees only a 3 × 3 patch and so can only respond to edges, colour contrasts and simple textures. Pooling and further convolutions enlarge

the region of the original image that a unit depends on (its receptive field). For the reference design the receptive field grows 3 → 8 → 18 pixels across the three convolutional layers, so the deepest convolutional units see an 18 × 18 region of the 32 × 32 image. They combine earlier features into corners, shapes and object parts, which the classifier maps to class scores.

4. Local connectivity and weight sharing. Local connectivity means that every output value depends only on a K × K window of the input. Weight sharing means that the same kernel weights are reused at every position. Together they make a convolutional layer far cheaper than a dense layer: the first convolutional layer maps 32 × 32 × 3 to 32 × 32 × 32 with only 896 parameters, whereas a dense layer producing the same 32,768 outputs from the 3,072 inputs would need (3,072 + 1) × 32,768 = 100,696,064 parameters. Weight sharing also lets the network detect a pattern wherever it appears in the image.

Q2. Convolution Output Shape [2 marks]

For one spatial dimension, the output size of a convolution is:

Output size = ⌊ (Input size + 2 × Padding − Kernel size) / Stride ⌋ + 1

The formula is applied **independently to the height and to the width**. If the input is H × W with a kernel of $K_h \times K_v$, strides S_h and S_v and paddings P_h and P_v, then the output height is $\lfloor (H + 2P_h - K_h)/S_h \rfloor + 1$ and the output width is $\lfloor (W + 2P_v - K_v)/S_v \rfloor + 1$. The output depth equals the number of filters F, regardless of the input depth. For square inputs and kernels the two calculations are identical, which is why one number is often quoted. The floor brackets mean that if the window does not fit exactly, the last partial position is discarded.

Calculate the output shape for each case. Show the formula, the substitution and the result for height and width separately.

Table 6: *Exercises for Q2*

Case	Input (H × W × C)	Kernel	Stride	Padding	Filters F	Output shape
(a)	32 × 32 × 3	5 × 5	1	0	16	28 × 28 × 16
(b)	64 × 64 × 3	3 × 3	2	1	24	32 × 32 × 24
(c)	40 × 60 × 8	5 × 5	2	2	32	20 × 30 × 32
(d)	100 × 100 × 1	7 × 7	3	0	12	32 × 32 × 12

1. Complete the table and show the working for each case.
2. For case (c), state how many rows and columns of the padded input are never covered by any kernel window, and why.
3. What padding value keeps the output size equal to the input size for a 5 × 5 kernel at stride 1? Give the general rule for an odd kernel size K.

Answer. 1. Completed table with working. The formula is applied to height and width separately.

Case	Height: working	Width: working	Output shape
(a)	⌊(32 + 2×0 − 5)/1⌋ + 1 = ⌊27/1⌋ + 1 = 27 + 1 = 28	⌊(32 + 2×0 − 5)/1⌋ + 1 = ⌊27/1⌋ + 1 = 27 + 1 = 28	28 × 28 × 16

Case	Height: working	Width: working	Output shape
(b)	$\lfloor(64 + 2\times1 - 3)/2\rfloor + 1 = \lfloor63/2\rfloor + 1 = 31.5\rightarrow31 + 1 = 32$	$\lfloor(64 + 2\times1 - 3)/2\rfloor + 1 = \lfloor63/2\rfloor + 1 = 31.5\rightarrow31 + 1 = 32$	$32 \times 32 \times 24$
(c)	$\lfloor(40 + 2\times2 - 5)/2\rfloor + 1 = \lfloor39/2\rfloor + 1 = 19.5\rightarrow19 + 1 = 20$	$\lfloor(60 + 2\times2 - 5)/2\rfloor + 1 = \lfloor59/2\rfloor + 1 = 29.5\rightarrow29 + 1 = 30$	$20 \times 30 \times 32$
(d)	$\lfloor(100 + 2\times0 - 7)/3\rfloor + 1 = \lfloor93/3\rfloor + 1 = 31 + 1 = 32$	$\lfloor(100 + 2\times0 - 7)/3\rfloor + 1 = \lfloor93/3\rfloor + 1 = 31 + 1 = 32$	$32 \times 32 \times 12$

2. Case (c). With stride 2 the windows land at positions 0, 2, 4, … of the padded input. In the height direction the padded size is 40 + 4 = 44, the last of the 20 windows ends at (20 − 1) × 2 + 5 = 43, so **1 padded row** is never covered. In the width direction the padded size is 64 and the last of the 30 windows ends at 63, so **1 padded column** is never covered. Neither dimension loses more than the other: both numerators (39 and 59) are odd, so the division by 2 leaves a remainder that the floor discards.

3. Padding that preserves the size. For a 5 × 5 kernel at stride 1 the output equals the input when (n + 2P − 5) + 1 = n, i.e. **P = 2**. In general, for an odd kernel size K and stride 1, P = (K − 1)/2 (P = 1 for K = 3, P = 2 for K = 5, P = 3 for K = 7).

Q3. Pooling Operations [1 mark]

Pooling slides a window over each feature map independently and replaces each window by one number. **Max pooling** keeps the largest value in the window; **average pooling** keeps the mean. Pooling has no trainable parameters and does not change the number of channels. Its output size uses the same formula as convolution with the pool size as kernel size (normally with zero padding).

1. Compare max pooling and average pooling: what each preserves, how each behaves with noise, and when you would prefer one over the other.
2. Apply 2 × 2 max pooling and 2 × 2 average pooling with stride 2 to the matrix below. Show the result of each window.
3. A 6 × 6 feature map is pooled with a 3 × 3 window and stride 3. What is the output size? What happens for a 7 × 7 map with the same settings?

Table 7: *Input feature map for Q3*

2	4	1	0
6	8	3	5
9	1	7	2
4	3	6	8

Answer. 1. Comparison. Max pooling keeps the strongest activation in each window, so it preserves the most distinctive feature response and is less affected by weak background values; it is the usual choice between convolutional layers when the presence of a feature matters more than its exact strength. Average pooling returns the mean of the window, which smooths noise and keeps information about the overall level of activation; it is often used at the end of a network (global average pooling) or when a smooth summary is wanted. Neither has trainable parameters, and both leave the number of channels unchanged.

Window (rows, cols)	Values	Max	Average
Top-left (1–2, 1–2)	2, 4, 6, 8	8	(2+4+6+8)/4 = 20/4 = 5.00
Top-right (1–2, 3–4)	1, 0, 3, 5	5	(1+0+3+5)/4 = 9/4 = 2.25
Bottom-left (3–4, 1–2)	9, 1, 4, 3	9	(9+1+4+3)/4 = 17/4 = 4.25
Bottom-right (3–4, 3–4)	7, 2, 6, 8	8	(7+2+6+8)/4 = 23/4 = 5.75

2. Results. The output size is $\lfloor (4 - 2)/2 \rfloor + 1 = 2$, so the output is 2 × 2. Max-pooled output: **[[8, 5], [9, 8]]**. Average-pooled output: **[[5.00, 2.25], [4.25, 5.75]]**.

3. Output sizes. For the 6 × 6 map: $\lfloor (6 - 3)/3 \rfloor + 1 = 1 + 1 = \mathbf{2}$, so the output is 2 × 2 and every input value is used exactly once. For the 7 × 7 map: $\lfloor (7 - 3)/3 \rfloor + 1 = \lfloor 1.33 \rfloor + 1 = \mathbf{2}$, again 2 × 2, but the last row and last column of the input are not covered by any window and are ignored.

Q4. Parameter Count [3 marks]

For a convolutional layer with biases:

$$\text{Parameters} = (K_h \times K_v \times C_{in} + 1) \times F$$

where C_{in} is the number of input channels, F is the number of filters and the "+ 1" is one bias per filter. For a dense layer with biases:

$$\text{Parameters} = (N_{in} + 1) \times N_{out}$$

where N_{in} is the number of input features and N_{out} is the number of output units. **Assumptions:** every convolutional and dense layer has a bias term; pooling, flatten, dropout and activation layers have no trainable parameters; batch-normalization layers are not used in this question. **Trainable parameters** (kernels, weights, biases) are changed by gradient descent. **Hyper-parameters** (kernel size, number of filters, learning rate, dropout rate) are chosen by you and are not learnt. **Non-trainable quantities** are values stored by the model but not updated by gradients, for example the running mean and variance of a batch-normalization layer.

Consider the following network for CIFAR-10. Find the output shape and the number of trainable parameters of every layer, then add the total.

Table 8: *Sample architecture for Q4 (the completed table is in the answer below)*

No.	Layer	Configuration
1	Input	32×32 RGB
2	Conv2D	16 filters, 5×5, stride 1, padding valid (0), ReLU
3	MaxPool2D	Max pooling, 2×2, stride 2
4	Conv2D	32 filters, 3×3, stride 1, padding valid (0), ReLU
5	MaxPool2D	Max pooling, 2×2, stride 2
6	Flatten	Reshape to a 1-D vector
7	Dense	64 units, ReLU
8	Dense	10 units, softmax

1. Complete the table and show the arithmetic for every layer that has parameters.
2. Which layer holds the largest share of the parameters? What does this tell you about where overfitting is most likely to arise?

3. If the 5 × 5 convolution is replaced by two stacked 3 × 3 convolutions (with 16 filters each, valid padding), how do the output shape after the pair and the parameter count change?

Answer. 1. Completed table. Shapes follow $\lfloor (n + 2P - K)/S \rfloor + 1$ with $P = 0$ for the convolutions and pooling; the Flatten output is $6 \times 6 \times 32 = 1{,}152$.

No.	Layer	Configuration	Output shape	Trainable parameters (working)
1	Input	32×32 RGB	32×32×3	—
2	Conv2D	16 filters, 5×5, stride 1, padding valid (0), ReLU	28×28×16	(5×5×3 + 1) × 16 = 76 × 16 = 1,216
3	MaxPool2D	Max pooling, 2×2, stride 2	14×14×16	0
4	Conv2D	32 filters, 3×3, stride 1, padding valid (0), ReLU	12×12×32	(3×3×16 + 1) × 32 = 145 × 32 = 4,640
5	MaxPool2D	Max pooling, 2×2, stride 2	6×6×32	0
6	Flatten	Reshape to a 1-D vector	1152	0
7	Dense	64 units, ReLU	64	(1,152 + 1) × 64 = 1,153 × 64 = 73,792
8	Dense	10 units, softmax	10	(64 + 1) × 10 = 65 × 10 = 650
	Total			**1,216 + 4,640 + 73,792 + 650 = 80,298**

2. Largest share. The first dense layer holds 73,792 of the 80,298 parameters, i.e. 73,792/80,298 = **91.9%**. The Flatten layer turns a small 6 × 6 × 32 map into 1,152 values and every one of them is connected to every one of the 64 units. Overfitting is therefore most likely to arise in this layer, which is why dropout and weight decay are normally applied to the dense part of the network.

3. Two stacked 3 × 3 convolutions instead of one 5 × 5. First 3 × 3 convolution (16 filters, valid): 32 → $\lfloor (32 - 3)/1 \rfloor + 1 = 30$, parameters (3×3×3 + 1) × 16 = 448. Second: 30 → 28, parameters (3×3×16 + 1) × 16 = 145 × 16 = 2,320. The output shape after the pair is **28 × 28 × 16**, the same as for the single 5 × 5 convolution, so all later layers are unchanged. The convolutional parameters rise from 1,216 to 448 + 2,320 = **2,768** (the total from 80,298 to 81,850), but the pair has the same 5 × 5 receptive field and an extra ReLU non-linearity.

Q5. Weight Initialization [2 marks]

Weights must start with small, random and different values: all-zero weights make every unit in a layer receive the same gradient, so the units never differentiate, and weights that are too large or too small make the signal and the gradients grow or shrink layer after layer. Two widely used schemes choose the spread of the random weights from the layer's **fan-in** (number of inputs to one unit) and **fan-out** (number of units fed by one input). For a convolutional layer, fan-in = $K_h \times K_v \times C_{in}$ and fan-out = $K_h \times K_v \times F$.

Table 9: *Xavier/Glorot and He initialization*

	Xavier / Glorot [4]	He / Kaiming [5]
Weight variance	2 / (fan-in + fan-out)	2 / fan-in
Common forms	Uniform on $\pm\sqrt{(6 / (\text{fan-in} + \text{fan-out}))}$, or normal with the variance above	Normal with standard deviation $\sqrt{(2 / \text{fan-in})}$
Designed for	Roughly linear or symmetric activations such as tanh and sigmoid	ReLU-family activations, which zero about half of the inputs
Keras names	glorot_uniform (Keras default), glorot_normal	he_normal, he_uniform

1. Explain in your own words why the factor of 2 in He initialization compensates for ReLU.
2. For the first convolutional layer of the reference design in Section 5, compute fan-in, fan-out, the He standard deviation and the Glorot variance.
3. Which initialization would you choose for a ReLU-based CNN? Justify the choice, and state what experiment would test it (Section 9, Experiment D).
4. Explain why biases are usually initialized to zero while weights are not.

Answer. 1. The factor 2 in He initialization. ReLU sets about half of the pre-activations to zero, so the variance of the signal passed to the next layer is roughly halved at every layer. With weight variance 1/fan-in the signal would shrink by about a factor 2 per layer; choosing Var(W) = 2/fan-in compensates for this halving, so that the variance of activations and of gradients stays about constant through the network. Glorot's 2/(fan-in + fan-out) assumes roughly linear (or symmetric) activations and has no such correction.

2. First convolutional layer of the reference design (3 × 3 kernel, 3 input channels, 32 filters): fan-in = 3 × 3 × 3 = **27**; fan-out = 3 × 3 × 32 = **288**. He standard deviation = $\sqrt{(2/27)} = \sqrt{0.0741} =$ **0.272**. Glorot variance = 2/(27 + 288) = 2/315 = **0.00635** (standard deviation 0.0797). He initialization therefore draws weights about 3.4 times wider than Glorot for this layer, which is appropriate because the layer has few inputs per unit and is followed by ReLU.

3. Choice. He (normal) initialization is chosen for all ReLU layers, because it is derived for ReLU and keeps the activation scale stable. The softmax output layer uses Glorot-uniform, since it is not followed by ReLU. This is tested by Experiment D, which repeats Experiment B with Glorot-uniform in every layer; the measured outcome is reported in Section 12.5.

4. Biases. Biases are initialized to zero because the symmetry between units is already broken by the random weights; a zero bias does not make units identical and does not shift the pre-activation range. Weights must not be zero: units with identical weights receive identical gradients and would remain identical.

Q6. Gradient-Based Optimization [3 marks]

Training minimises the loss L(w) over the parameters w. **Backpropagation** applies the chain rule layer by layer from the output to the input to compute the gradient $g = \partial L/\partial w$ of every parameter. **Gradient descent** then moves each parameter against its gradient. The size of the step is set by the **learning rate** η:

$$w \leftarrow w - \eta \cdot g$$

In **mini-batch** training the gradient is estimated from a small batch of images (for example 64), which is cheaper and noisier than using the full training set. The role of an **optimizer** is to decide how the gradient is turned into an update. Two common choices are:

Table 10: *Two common optimizers*

	SGD with momentum	Adam [6]
Update rule	$v \leftarrow \beta v + g$; $w \leftarrow w - \eta v$ (β is often 0.9)	$m \leftarrow \beta_1 m + (1-\beta_1)g$; $v \leftarrow \beta_2 v + (1-\beta_2)g^2$; $\hat{m} = m/(1-\beta_1^t)$, $\hat{v} = v/(1-\beta_2^t)$; $w \leftarrow w - \eta\, \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$
Idea	Accumulates a velocity that smooths noisy gradients and speeds movement along consistent directions [9]	Keeps running averages of the gradient and its square and scales each parameter's step individually

	SGD with momentum	Adam [6]
Typical setting	η around 0.01–0.1; needs more learning-rate tuning	η around 0.001 as a starting point; $\beta_1 = 0.9$, $\beta_2 = 0.999$; ε is a small constant
Trade-off	Often generalises well when well tuned, but is more sensitive to η	Usually converges quickly with little tuning, but can generalise differently from SGD

1. Explain gradient descent, backpropagation and the learning rate in your own words, with a short numerical example of one weight update.
2. Describe what happens if the learning rate is too large and if it is too small, and how each appears in the loss curve.
3. Choose an optimizer for the experiments and **justify the choice from the viewpoint of this problem**. Do not claim that one optimizer is always better; the best choice depends on the model, the data and the tuning effort.

Answer. 1. Gradient descent, backpropagation and learning rate. Training minimises the loss $L(w)$. Backpropagation applies the chain rule from the output back to the input and gives $g = \partial L/\partial w$ for every weight. Gradient descent changes each weight against its gradient, $w \leftarrow w - \eta\, g$, where the learning rate η sets the step size. Example (Section 6.7): for $w = 0.5$, $g = 0.8$ and $\eta = 0.1$ the new weight is $0.5 - 0.1 \times 0.8 = 0.42$.

2. Learning rate too large or too small. If η is too large the steps overshoot the minimum: the loss oscillates, jumps up, or becomes NaN (Scenario Z in Q9). If η is too small the loss falls very slowly and training may stop (through early stopping or the epoch limit) far from a good solution; in the curves this appears as a smooth, almost flat, decrease.

3. Choice of optimizer. Adam with learning rate 0.001 is the main optimizer. The network has parameters of very different gradient scales (small 3 × 3 kernels and a 262,272-parameter dense layer), and Adam adapts the step of every parameter, so it needs little tuning — which matters when the training budget is at most 30 epochs and the learning rate cannot be searched. The evidence from Experiments B and C supports the choice on speed, not on final accuracy: after epoch 3 the validation accuracy was 66.6% with Adam and 60.2% with SGD plus momentum, but at the best epoch the two were similar (validation 72.4% against 72.1%, test 71.9% against 72.4%). SGD with momentum is therefore not worse here, and a differently tuned learning rate might have changed the comparison; the claim that one optimizer is always better is not supported by these results.

Q7. Regularization [2 marks]

A model **overfits** when it learns details of the training images that do not generalise, so training accuracy keeps improving while validation accuracy stalls or falls. Regularization techniques reduce this gap.

1. **Dropout** [7]: during training each unit is set to zero with probability p and the others are scaled by $1/(1-p)$; at inference all units are used. Explain why this discourages units from co-adapting.
2. **L2 regularization / weight decay**: a penalty is added to the loss. In the Keras convention, $\text{Loss_total} = \text{Loss_data} + \lambda \Sigma\, w^2$. Explain how this keeps the weights small and why that tends to give smoother decision functions.
3. **Data augmentation**: random label-preserving changes (for example horizontal flips and small shifts) are applied to training images only. Explain why this acts as a regularizer and why it must not be applied to validation or test images.

4. **Early stopping**: training is stopped when the validation loss has not improved for a number of epochs (the patience) and the best weights are restored. Explain what this assumes and why the **test** set must not be used for it.
5. For each technique, state one possible disadvantage or cost.

Answer. 1. Dropout. During training each unit of the 128-unit dense layer is set to zero with probability 0.5 and the remaining units are scaled by $1/(1 - 0.5) = 2$; at inference all units are used. A unit can then not rely on the presence of particular other units, which prevents co-adaptation and makes the layer behave like an average of many thinned networks. In the experiments dropout was used together with L2 (Experiment B); it reduced the accuracy gap at the best epoch from 7.1 to 2.5 points and delayed overfitting (best epoch 8 against 5). **Cost:** training is noisier and slower to converge, and its rate is one more hyper-parameter.

2. L2 weight decay. The penalty $\lambda \Sigma w^2$ with $\lambda = 1\times10^{-4}$ is added to the loss, so large weights are penalized and the weights stay small, which gives smoother functions that react less to individual training images. It was used in Experiments B–E together with dropout, so its separate effect was not isolated. **Cost:** a biased solution if λ is too large, and the reported loss includes the penalty, so the losses of B–E are not directly comparable with the loss of A (the accuracies are).

3. Data augmentation. Random horizontal flips and shifts of up to 10% create new, label-preserving training images each epoch, so the network cannot memorize exact pixels. It is the most effective technique in this study: Experiment E (B plus augmentation) raised validation accuracy from 72.4% to 78.0% (+5.6 points) and test accuracy from 71.9% to 77.7%. It is applied to training images only, because validation and test images must remain the unchanged data on which generalization is measured. **Cost:** much longer training (826 s against 418 s for B, as the model needed 24 epochs instead of 13) and the risk of augmentations that do not suit the data.

4. Early stopping. Training stops when validation loss has not improved for 5 epochs and the weights of the best epoch are restored. It assumes that the validation loss is a good proxy for generalization and that it is not rising only temporarily. Its value is clear in Experiment A: validation loss was lowest (0.8251) at epoch 5 and had risen to 1.2690 by epoch 10, while training accuracy kept rising; restoring epoch 5 avoided that. The test set must not be used for this decision, because the stopping point would then be tuned to the test data and the test result would be optimistic.

5. Costs. Dropout: slower and noisier training. L2: bias if λ is too large and an extra hyper-parameter. Augmentation: longer training and the need for sensible transformations. Early stopping: needs a validation set (45,000 instead of 50,000 training images here) and a patience value.

Q8. Loss Functions and Evaluation Metrics [1 mark]

The softmax layer turns the 10 raw outputs (logits) z into probabilities $p_k = e^{z_k} / \Sigma_j e^{z_j}$. **Sparse categorical cross-entropy** compares the predicted probabilities with the integer true label y (**categorical** cross-entropy is the same quantity for one-hot labels):

$$L = -(1/N) \Sigma_i \log p_i[y_i]$$

Only the probability given to the correct class matters; the loss is 0 when that probability is 1 and grows without bound as it approaches 0. For the multi-class problem the following metrics are computed from the confusion matrix, whose entry C[i][j] is the number of test images with true class i predicted as class j:

Accuracy = (number correct) / (number of samples)

$\text{Precision}^c = TP^c / (TP^c + FP^c)$ $\quad$ $\text{Recall}^c = TP^c / (TP^c + FN^c)$

$F1^c = 2 \cdot Precision^c \cdot Recall^c / (Precision^c + Recall^c)$

1. Explain each term and why cross-entropy rather than accuracy is used as the training objective.
2. A 10-class network that outputs equal probabilities for all classes has what loss? Use this to state the loss you expect at the very start of training.
3. Explain macro-averaged precision, recall and F1, and give an example in which accuracy alone would be misleading.
4. Explain how to read a confusion matrix, including which cells show the most common mistakes.

Answer. 1. Terms and why cross-entropy. The softmax turns the 10 logits into probabilities that sum to 1. Sparse categorical cross-entropy is $-(1/N)\ \Sigma \log p[y]$, the average negative log-probability that the network gives to the correct class; precision, recall and F1 are defined in the formulas above from true positives (TP), false positives (FP) and false negatives (FN) of each class. Cross-entropy is used as the **training objective** because it is smooth and differentiable and gives informative gradients for every image, even when the arg-max prediction is unchanged; accuracy is a step function whose gradient is zero almost everywhere, so it cannot be optimized by gradient descent.

2. Initial loss. A network that outputs probability 1/10 for every class has loss $-\ln(0.1) = \ln 10 =$ **2.3026**, the loss expected for a pure guess. The measured loss of the untrained reference network on 1,000 validation images was **3.4577**, higher than 2.3026, because He initialization produces logits that are not nearly equal: their spread (about 1.78) adds roughly $\text{spread}^2/2$ to ln 10. A starting loss of exactly ln 10 should therefore be expected only when the output layer starts with very small weights.

3. Macro averages and the limits of accuracy. Macro-averaged precision, recall and F1 are the unweighted means of the ten per-class values, so every class counts equally. For the final model they are 0.7769, 0.7768 and 0.7737. Accuracy alone would mislead on an imbalanced set: a model that always predicts the majority class of a 99%/1% problem has 99% accuracy and no value. Even on this balanced data the single figure 77.7% hides large differences between classes: F1 is 0.8898 for automobile but only 0.6189 for cat.

4. Reading the confusion matrix. Row i is the true class and column j the predicted class; the diagonal holds the correct predictions and each off-diagonal cell counts one kind of mistake. For the final model the largest mistakes are **dog predicted as cat (156 images)**, cat as dog (105), cat as frog (102), bird as frog (95) and deer as horse (94): classes with similar shape, colour or background are confused.

Q9. Overfitting and Underfitting [1 mark]

The following **hypothetical** numbers are given only for practice. They are not results of any real experiment.

Table 11: *Hypothetical training logs for interpretation*

Scenario	Train acc.	Val. acc.	Train loss	Val. loss	Observation during training
X	99%	70%	0.03	1.60	Validation loss reached a minimum near epoch 8 and then kept rising
Y	48%	47%	1.45	1.47	Both curves flattened early at a poor level
Z	10%	10%	NaN	NaN	Loss jumped upward within the first few updates after a large learning rate was set

1. For each scenario, state whether the model is overfitting, underfitting or training unstably, and give the evidence.

2. Suggest at least two concrete improvements for each scenario and say what you would expect to see in the learning curves afterwards.

Answer. 1. Diagnosis.

Scenario	Diagnosis	Evidence
X	**Overfitting**	Training accuracy 99% against validation 70% (gap 29 points); training loss 0.03 while validation loss is 1.60 and rose after epoch 8
Y	**Underfitting**	Both accuracies are low (48% and 47%) and almost equal; both losses are high and flat, so the model cannot fit even the training data
Z	**Unstable training (diverging)**	Accuracy 10% equals chance for 10 classes; loss became NaN soon after a large learning rate was set

2. Improvements and expected effect.

X: (i) add dropout and L2 weight decay, or raise them; (ii) add data augmentation (flips, shifts); (iii) stop at the epoch with the lowest validation loss (early stopping). Expected: training accuracy falls a little, validation accuracy rises, and the two curves move closer together.

Y: (i) increase capacity (more filters or another convolutional block); (ii) train longer or raise the learning rate; (iii) reduce regularization. Expected: training accuracy and loss improve first, and validation follows.

Z: (i) reduce the learning rate by 10× or more; (ii) check that inputs are normalized; (iii) use gradient clipping or a smaller batch step. Expected: the loss decreases smoothly from about ln 10 = 2.30 instead of jumping.

Q10. Experimental Analysis [2 marks]

Using the experiments of Section 9, compare **at least two controlled configurations** — for example, the same network trained with two different optimizers, or with and without a regularization technique. A comparison is controlled only if everything that is not under study (architecture, data split, batch size, seed, number of epochs or the early-stopping rule) is held fixed.

1. State which single factor was changed and which factors were kept fixed.
2. Compare the validation and test results, the train–validation gap and the speed of convergence.
3. Explain the differences using the theory of Q5–Q7 and say whether the evidence is strong or weak. A single run with one seed cannot show that one method is better in general; state this limitation and, if time allows, repeat with several seeds and report the spread.

Answer. 1. Factors changed and kept fixed. *Comparison 1 (regularization), A → B:* only dropout 0.5 and L2 1×10^{-4} were added. *Comparison 2 (optimizer), B → C:* only the optimizer (Adam 0.001 → SGD with momentum 0.9, learning rate 0.01) changed. *Comparison 3 (augmentation), B → E:* only augmentation was added. In all three, the architecture, the data split, the seed, the batch size (64) and the early-stopping rule were fixed.

Comparison	Val. acc.	Test acc.	Train–val. gap	Epochs (best)
A baseline	72.4%	71.2%	7.1 points	10 (5)
B = A + dropout/L2	72.4%	71.9%	2.5 points	13 (8)

Comparison	Val. acc.	Test acc.	Train–val. gap	Epochs (best)
C = B with SGD + momentum	72.1%	72.4%	1.8 points	16 (11)
E = B + augmentation	78.0%	77.7%	-2.8 points	24 (19)

2. Comparison. *A → B:* validation accuracy was unchanged (72.4% against 72.4%), test accuracy rose by 0.7 points, the gap at the best epoch fell from 7.1 to 2.5 points and the best epoch moved from 5 to 8. *B → C:* SGD converged more slowly at first (validation accuracy 60.2% against 66.6% after epoch 3) and needed 16 epochs against 13, but ended at a similar validation accuracy (72.1% against 72.4%) and test accuracy (72.4% against 71.9%). *B → E:* validation accuracy rose by 5.6 points and test accuracy by 5.8 points, and the training accuracy (75.2%) fell below the validation accuracy because the training metric is measured on augmented, dropout-perturbed images.

3. Explanation and strength of evidence. Dropout and weight decay act on the dense layer where most parameters lie (Q4, Q7), so they narrow the train–validation gap and delay overfitting, but with the same network and data they did not by themselves raise the best validation accuracy. Augmentation adds new information to the training set, and its effect (5.6 points) is much larger than the differences among A, B and C (all within 0.3 points on validation), so that evidence is **strong**; the A/B/C differences are **weak**. Each comparison is **one run with one seed**: a different seed could change differences of about one point, so these results do not show that one optimizer or regularizer is better in general. The learning rate also changed in Comparison 2, and the losses of B, C and E contain the L2 penalty and so cannot be compared with the loss of A.

5. CNN Architecture Design Task

5.1 The design problem

Propose a CNN for the CIFAR-10 problem of Section 2. Your design must be small enough to train in reasonable time on your hardware, and every number in your design must be calculated by hand **before** you train the model and checked against the framework's model summary afterwards.

Your architecture must contain:

- an **input layer** of shape (32, 32, 3);
- at least **two convolutional layers** with a stated kernel size, number of filters, stride and padding;
- an **activation function** after each convolution and hidden dense layer (ReLU is recommended);
- **pooling layers** (max or average) between convolutional blocks;
- **flattening or global average pooling** to connect to the classifier;
- a **fully connected hidden layer (optional) and a 10-unit softmax output layer**;
- a stated **weight initialization**, **optimizer**, **loss function** and **regularization techniques**.

You may adopt the reference design below, but you are encouraged to change at least the number of filters, the depth or the dense-layer width and to justify your changes. If you adopt it unchanged, explain why each choice is reasonable. The reference design is an **illustration, not the only acceptable solution.**

5.2 Reference design

The reference design uses three convolution–pooling blocks with 3 × 3 kernels, stride 1 and "same" padding (padding 1, which keeps height and width unchanged), followed by one hidden dense layer with dropout and a softmax output layer. Padding 1 for a 3 × 3 kernel gives, for example, (32 + 2×1 − 3)/1 + 1 = 32. Activation functions are shown inside the layer they follow. The table shows the shapes and parameter counts that Section 6 explains step by step.

Table 12: *Reference architecture (illustrative; all shapes and counts calculated with the formulas of Section 4)*

Layer No.	Layer Type	Input Shape	Configuration	Output Shape	Trainable Parameters
1	Input	—	32×32 RGB image	32×32×3	0
2	Conv2D	32×32×3	32 filters, 3×3, stride 1, padding same (1), ReLU	32×32×32	896
3	MaxPool2D	32×32×32	Max pooling, 2×2, stride 2	16×16×32	0
4	Conv2D	16×16×32	64 filters, 3×3, stride 1, padding same (1), ReLU	16×16×64	18,496
5	MaxPool2D	16×16×64	Max pooling, 2×2, stride 2	8×8×64	0
6	Conv2D	8×8×64	128 filters, 3×3, stride 1, padding same (1), ReLU	8×8×128	73,856
7	MaxPool2D	8×8×128	Max pooling, 2×2, stride 2	4×4×128	0
8	Flatten	4×4×128	Reshape to a 1-D vector	2048	0
9	Dense	2048	128 units, ReLU	128	262,272
10	Dropout	128	rate 0.5 (training only)	128	0
11	Dense	128	10 units, softmax	10	1,290
	Total				**356,810**

Training settings for the reference design (proposed): He-normal initialization for the convolutional and hidden dense layers (Glorot-uniform for the softmax output layer), sparse categorical cross-entropy loss, Adam with learning rate 0.001 as the baseline optimizer, batch size 64, at most 30 epochs with early stopping on validation loss (patience 5), dropout 0.5 and L2 weight decay 1×10^{-4} in the regularized experiments.

Design rationale and notes

- Doubling the filters (32 → 64 → 128) while halving the spatial size (32 → 16 → 8 → 4) is a common pattern: spatial detail shrinks, while the number of learnt feature types grows.
- The Flatten layer turns 4 × 4 × 128 = 2,048 values into one vector. The first dense layer then holds 262,272 of the 356,810 parameters, about 73.5% of the model, which is why dropout is placed after it.
- All **356,810** parameters are trainable. There are no non-trainable parameters because the design has no batch-normalization layer.
- Pooling, Flatten and Dropout layers have 0 parameters. Dropout acts only during training; the layer exists at inference but passes values through unchanged.
- Replacing Flatten with global average pooling would turn the 4 × 4 × 128 map into 128 values and cut the first dense layer to (128 + 1) × 128 = 16,512 parameters. This is a design alternative you may explore.

5.3 Submitted architecture and verification

The reference design of Section 5.2 is adopted **unchanged** as the architecture submitted for [4 marks] (output shapes) and [4 marks] (parameter counts). The design choices are justified as follows:

- **3 × 3 kernels with "same" padding** keep the shape arithmetic simple, and three stacked 3 × 3 convolutions with two 2 × 2 poolings give the deepest convolutional units an 18 × 18 receptive field (Q1), which is more than half of a 32 × 32 image.
- **Filters 32 → 64 → 128 while the size falls 32 → 16 → 8 → 4** keeps the cost of the convolutional layers balanced: the multiply–accumulate counts are 32×32×32×27 = 884,736, then 16×16×64×288 = 4,718,592 and 8×8×128×576 = 4,718,592, about 10.6 million operations per image including the dense layers.
- **Small enough for a CPU:** the measured training time is about 27–30 seconds per epoch on two CPU cores (Section 9).
- **Training choices:** He-normal initialization for the ReLU layers and Glorot-uniform for the softmax layer, sparse categorical cross-entropy, Adam (learning rate 0.001), and dropout 0.5 with L2 weight decay 1×10^{-4} in the regularized experiments.

Every shape and parameter count of the reference architecture in Section 5.2 was calculated by hand before training and compared with the layers of the Keras model (tests T05 and T06 of Section 11). The comparison is shown below; all values agree.

Table 13: *Hand calculation compared with the framework's model summary*

No.	Layer	Output shape (hand)	Output shape (Keras)	Parameters (hand)	Parameters (Keras)	Match
2	Conv2D	32×32×32	32×32×32	896	896	Yes
3	MaxPool2D	16×16×32	16×16×32	0	0	Yes
4	Conv2D	16×16×64	16×16×64	18,496	18,496	Yes
5	MaxPool2D	8×8×64	8×8×64	0	0	Yes
6	Conv2D	8×8×128	8×8×128	73,856	73,856	Yes
7	MaxPool2D	4×4×128	4×4×128	0	0	Yes
8	Flatten	2048	2048	0	0	Yes
9	Dense	128	128	262,272	262,272	Yes
10	Dropout	128	128	0	0	Yes
11	Dense	10	10	1,290	1,290	Yes
	Total			**356,810**	**356,810**	**Yes**

Keras reports 356,810 trainable and 0 non-trainable parameters (Listing in Section 7.4).

6. Mathematical Calculations

This section shows the calculations in full. The numbers are deliberately **different** from those in the exercises of Section 4, so you can check your method but not copy the answers.

6.1 Convolution output dimensions

> Output size = ⌊ (Input + 2P − K) / S ⌋ + 1

Table 14: *Worked examples: convolution output size*

Case	Input H × W	K, S, P	Height: substitution and result	Width: substitution and result	Output H × W
A	28 × 28	3, 1, 0	⌊(28 + 2×0 − 3) / 1⌋ + 1 = ⌊25/1⌋ + 1 = 25 + 1 = 26	⌊(28 + 2×0 − 3) / 1⌋ + 1 = ⌊25/1⌋ + 1 = 25 + 1 = 26	26 × 26
B	32 × 32	3, 1, 1	⌊(32 + 2×1 − 3) / 1⌋ + 1 = ⌊31/1⌋ + 1 = 31 + 1 = 32	⌊(32 + 2×1 − 3) / 1⌋ + 1 = ⌊31/1⌋ + 1 = 31 + 1 = 32	32 × 32
C	32 × 32	5, 2, 2	⌊(32 + 2×2 − 5) / 2⌋ + 1 = ⌊31/2⌋ + 1 = 15.5→15 + 1 = 16	⌊(32 + 2×2 − 5) / 2⌋ + 1 = ⌊31/2⌋ + 1 = 15.5→15 + 1 = 16	16 × 16
D	64 × 48	3, 2, 1	⌊(64 + 2×1 − 3) / 2⌋ + 1 = ⌊63/2⌋ + 1 = 31.5→31 + 1 = 32	⌊(48 + 2×1 − 3) / 2⌋ + 1 = ⌊47/2⌋ + 1 = 23.5→23 + 1 = 24	32 × 24
E	224 × 224	7, 2, 3	⌊(224 + 2×3 − 7) / 2⌋ + 1 = ⌊223/2⌋ + 1 = 111.5→111 + 1 = 112	⌊(224 + 2×3 − 7) / 2⌋ + 1 = ⌊223/2⌋ + 1 = 111.5→111 + 1 = 112	112 × 112

Interpretation. Case A shrinks the image by K − 1 = 2 pixels because no padding is used. Case B uses padding 1 and keeps the size. Case C uses stride 2, which roughly halves the size; the exact value 15.5 is rounded down, so one partial window position at the border is lost. Case D has different height and width, so the two dimensions must be calculated separately (63/2 = 31.5 and 47/2 = 23.5). Case E is the first layer of many ImageNet-size networks. The number of filters sets the output depth only; it does not enter the formula.

6.2 Effects of stride and padding

The table varies stride and padding for one fixed input (32 × 32) and kernel (3 × 3).

Table 15: *Effect of stride and padding on output size (input 32, kernel 3)*

Stride	Padding	Calculation	Output size	Effect
1	0	⌊(32 + 2×0 − 3) / 1⌋ + 1 = ⌊29/1⌋ + 1 = 29 + 1 = 30	30	Border pixels are lost; size shrinks by 2
1	1	⌊(32 + 2×1 − 3) / 1⌋ + 1 = ⌊31/1⌋ + 1 = 31 + 1 = 32	32	Size preserved (“same”)
2	0	⌊(32 + 2×0 − 3) / 2⌋ + 1 = ⌊29/2⌋ + 1 = 14.5→14 + 1 = 15	15	About half the size; partial window dropped
2	1	⌊(32 + 2×1 − 3) / 2⌋ + 1 = ⌊31/2⌋ + 1 = 15.5→15 + 1 = 16	16	Exactly halved; border retained

Reading the table. Padding adds a border of zeros, so the kernel can be centred on edge pixels; this keeps information at the border and allows many convolutions to be stacked without the image vanishing. For an odd kernel size K and stride 1, padding P = (K − 1)/2 preserves the size. Stride controls how far the window moves: stride 2 reduces the height and width by about half, cutting later computation by about four times, at the cost of skipping positions.

6.3 Pooling output dimensions and a manual pooling example

Pooling uses the same formula with kernel size = pool size and padding 0. For a 2 × 2 pool with stride 2: a 32 × 32 map gives $\lfloor (32 - 2)/2 \rfloor + 1 = 16$; an odd 13 × 13 map gives $\lfloor (13 - 2)/2 \rfloor + 1 = \lfloor 5.5 \rfloor + 1 = 6$, so the last row and column are ignored. The following 4 × 4 map is pooled with a 2 × 2 window and stride 2, giving a 2 × 2 output ($\lfloor (4 - 2)/2 \rfloor + 1 = 2$).

1	3	2	4
5	6	1	2
7	2	9	0
3	4	1	8

Table 16: *Window-by-window pooling result*

Window (rows, cols)	Values	Max	Average
Top-left (1–2, 1–2)	1, 3, 5, 6	6	(1+3+5+6)/4 = 15/4 = 3.75
Top-right (1–2, 3–4)	2, 4, 1, 2	4	(2+4+1+2)/4 = 9/4 = 2.25
Bottom-left (3–4, 1–2)	7, 2, 3, 4	7	(7+2+3+4)/4 = 16/4 = 4.00
Bottom-right (3–4, 3–4)	9, 0, 1, 8	9	(9+0+1+8)/4 = 18/4 = 4.50

Max-pooled output: [[6, 4], [7, 9]]. Average-pooled output: [[3.75, 2.25], [4.00, 4.50]]. Max pooling keeps the strongest response in each window; average pooling smooths it.

6.4 Number of convolutional parameters

> Parameters = $(K_h \times K_v \times C_{in} + 1) \times F$

Table 17: *Worked examples: convolutional layer parameters*

Layer	Substitution	Weights	Biases	Total
3×3 kernel, 3 in, 32 filters	(3×3×3 + 1) × 32 = 28 × 32	864	32	896
5×5 kernel, 16 in, 24 filters	(5×5×16 + 1) × 24 = 401 × 24	9,600	24	9,624
1×1 kernel, 64 in, 32 filters	(1×1×64 + 1) × 32 = 65 × 32	2,048	32	2,080

Check of the first row: weights = 3 × 3 × 3 × 32 = 864 and biases = 32, so 864 + 32 = 896. Note that the count does **not** depend on the image height or width; this is the effect of weight sharing.

6.5 Number of dense-layer parameters

> Parameters = $(N_{in} + 1) \times N_{out}$

Layer	Substitution	Weights	Biases	Total
Dense, 2,048 in, 128 out	(2,048 + 1) × 128 = 2,049 × 128	262,144	128	262,272
Dense, 128 in, 10 out	(128 + 1) × 10 = 129 × 10	1,280	10	1,290

A dense layer connects every input to every output, so for images it dominates the parameter count: the first dense layer above has 262,272 parameters, more than three times the 73,856 of the largest convolutional layer in the reference design.

6.6 Total trainable parameters: a complete worked example

Consider a small network for 28 × 28 grey-scale images (1 channel) and 10 classes: Conv(8 filters, 3×3, valid) → MaxPool(2×2) → Conv(16 filters, 3×3, valid) → MaxPool(2×2) → Flatten → Dense(32, ReLU) → Dense(10, softmax).

Table 18: *Step-by-step shapes and parameters of the small example network*

No.	Layer	Output shape	Calculation	Parameters
1	Input	28×28×1	—	0
2	Conv2D	26×26×8	$\lfloor(28-3)/1\rfloor+1=26$; $(3\times3\times1+1)\times8=10\times8=80$	80
3	MaxPool2D	13×13×8	$\lfloor(26-2)/2\rfloor+1=13$; no parameters	0
4	Conv2D	11×11×16	$\lfloor(13-3)/1\rfloor+1=11$; $(3\times3\times8+1)\times16=73\times16=1{,}168$	1,168
5	MaxPool2D	5×5×16	$\lfloor(11-2)/2\rfloor+1=\lfloor4.5\rfloor+1=5$; no parameters	0
6	Flatten	400	$5\times5\times16=400$; no parameters	0
7	Dense	32	$(400+1)\times32=12{,}832$	12,832
8	Dense	10	$(32+1)\times10=330$	330
	Total		**80 + 1,168 + 12,832 + 330**	**14,410**

Interpretation. The total is 14,410 trainable parameters. 89% of them (12,832 of 14,410) are in the first dense layer, although it is a small layer, because Flatten produced 400 inputs for each of its 32 units. Both convolutional layers together hold only 1,248 parameters (8.7%).

6.7 Basic gradient-descent weight updates

Take one weight $w = 0.5$ with learning rate $\eta = 0.1$, and suppose backpropagation gives the gradient $g = \partial L/\partial w = 0.8$.

Plain gradient descent: $w_1 = w - \eta\, g = 0.5 - 0.1 \times 0.8 = 0.5 - 0.08 = 0.42$

A positive gradient means that increasing w increases the loss, so w is decreased. **SGD with momentum** ($\beta = 0.9$, initial velocity $v_0 = 0$) with a second gradient $g_2 = 0.6$:

$v_1 = \beta\, v_0 + g_1 = 0.9 \times 0 + 0.8 = 0.8 \qquad w_1 = 0.5 - 0.1 \times 0.8 = 0.42$

$v_2 = \beta\, v_1 + g_2 = 0.9 \times 0.8 + 0.6 = 1.32 \qquad w_2 = 0.42 - 0.1 \times 1.32 = 0.42 - 0.132 = 0.288$

Because the two gradients point the same way, the velocity grows and the second step (0.132) is larger than the first (0.08). With plain gradient descent the second step would be $0.1 \times 0.6 = 0.06$.

Adam, first step ($\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $g = 0.8$, $m_0 = v_0 = 0$):

$m_1 = 0.1 \times 0.8 = 0.08; \qquad v_1 = 0.001 \times 0.64 = 0.00064$

$\hat{m} = 0.08/0.1 = 0.8; \qquad \hat{v} = 0.00064/0.001 = 0.64$

$\Delta w = -\eta\, \hat{m}/(\sqrt{\hat{v}} + \varepsilon) = -0.001 \times 0.8/(0.8 + \varepsilon) \approx -0.001, \qquad w_1 \approx 0.499$

The first Adam step has a size of about η whatever the size of the gradient, which is one reason Adam is less sensitive to the gradient scale.

6.8 Interpretation of loss reduction during training

Cross-entropy for one image is $-\ln p$, where p is the probability given to the correct class:

Table 19: *Per-image cross-entropy as a function of the probability of the true class*

Probability p of correct class	Loss = −ln p	Meaning
0.10 (a random guess among 10 classes)	2.3026	No knowledge; expected loss at the start of training is ln 10 = 2.3026
0.50	0.6931	Moderately confident and right
0.90	0.1054	Confident and right
0.01	4.6052	Confident and wrong: the loss is large

Reading a loss reduction. Suppose, purely as an arithmetic illustration, that the average training loss falls from 2.10 in epoch 1 to 1.15 in epoch 5. The relative reduction is (2.10 − 1.15)/2.10 = 0.95/2.10 ≈ 0.452, a drop of about 45%. Since a loss of L corresponds to a geometric-mean probability of e^(−L) on the correct class, the model has moved from about e^(−2.10) ≈ 0.12 to e^(−1.15) ≈ 0.32. Loss can fall while accuracy stays flat, because confidence can improve before the arg-max prediction changes. A falling training loss with a **rising validation loss** signals overfitting (Section 10).

6.9 Practice exercises [3 marks]

The exercises and their solutions with full working follow.

1. A weight w = −0.4 has gradient g = 1.5. With plain gradient descent and η = 0.05, find the new weight.
2. With SGD momentum (β = 0.9, η = 0.1, $v_0 = 0$), a weight starts at $w_0 = 1.0$ and receives gradients $g_1 = 1.0$ and $g_2 = 0.5$. Find v_1, w_1, v_2 and w_2.
3. A 3-class network produces logits z = [2.0, 1.0, 0.1] for an image whose true class is class 1 (the first one). Compute the three softmax probabilities and the cross-entropy loss.
4. The average training loss of a 10-class network falls from 2.30 to 0.90. Compute the relative reduction, the corresponding geometric-mean probability on the correct class before and after, and comment on whether this tells you anything about generalization.

Answer. 1. $w_1 = w - \eta\, g = -0.4 - 0.05 \times 1.5 = -0.4 - 0.075 =$ **−0.475**.

2. $v_1 = \beta\, v_0 + g_1 = 0.9 \times 0 + 1.0 =$ **1.0**; $w_1 = 1.0 - 0.1 \times 1.0 =$ **0.90**. $v_2 = 0.9 \times 1.0 + 0.5 =$ **1.4**; $w_2 = 0.90 - 0.1 \times 1.4 = 0.90 - 0.14 =$ **0.76**.

3. e^2.0 = 7.3891, e^1.0 = 2.7183, e^0.1 = 1.1052; sum = 11.2125. Probabilities: $p_1 = 7.3891/11.2125 =$ **0.6590**, $p_2 = 2.7183/11.2125 =$ **0.2424**, $p_3 = 1.1052/11.2125 =$ **0.0986** (sum 1.0000). The true class is class 1, so the loss is −ln 0.6590 = **0.4170**.

4. Relative reduction = (2.30 − 0.90)/2.30 = 1.40/2.30 = 0.609, i.e. about **60.9%**. The geometric-mean probability on the correct class rises from e^(−2.30) = **0.100** (a random guess among 10 classes) to e^(−0.90) = **0.407**. This describes the **training** loss only: it shows that the network fits the training images better, but says nothing about generalization. Only the validation loss and the test results can show whether the improvement carries over to unseen images.

7. Implementation Requirements

7.1 Framework and environment

Implement the assignment in **Python with TensorFlow/Keras**, which is the primary path used for all code and for the reference listings below. If you prefer PyTorch you may use it, but every item in Section 7.2 must still be covered and the architecture, shapes and parameter counts must be unchanged. Using the equivalent PyTorch classes (nn.Conv2d, nn.MaxPool2d, nn.Linear, nn.CrossEntropyLoss, torch.optim) is straightforward and is documented in the official PyTorch documentation [12].

Table 20: *Suggested environment (record your actual versions in the report)*

Item	Suggestion
Python	3.9 or later
Libraries	tensorflow (includes Keras), numpy, matplotlib, scikit-learn
Hardware	CPU is enough for the reference design; a GPU shortens training
Files	train_cnn.py (code), requirements.txt (versions of the libraries actually used), and the generated results/ folder

7.2 What the implementation must cover [3 marks]

1. Load the dataset.
2. Inspect image shapes, data types and class labels.
3. Pre-process and normalize the images.
4. Create training, validation and test splits appropriately.
5. Build the CNN architecture.
6. Select a suitable weight initializer.
7. Compile or configure the model with the chosen optimizer and loss function.
8. Apply regularization.
9. Train the network.
10. Save the training history.
11. Plot training and validation accuracy.
12. Plot training and validation loss.
13. Evaluate the final model on the test dataset.
14. Generate a confusion matrix and a classification report.
15. Record the final model configuration and results.

Rules for a fair evaluation

• **The test set must not be used to tune anything**: not the architecture, learning rate, regularization strength, number of epochs, or the early-stopping decision. These decisions use the validation set only. Evaluate on the test set once, after the final configuration is fixed.

• **Normalization must not leak test information.** Compute the channel means and standard deviations on the **training split only**, then apply those same numbers to the validation and test images.

• **Data augmentation is applied to training images only**, and is applied after the split so augmented copies of one image never appear in two different splits.

• Split by index before any processing, and check that the three index sets do not overlap.

The listings that follow are a **reference implementation to start from** (written against TensorFlow/Keras 2.x and 3.x conventions). Adapt it to your own architecture and verify it on your own system; the numbers it produces are yours to report.

7.3 Reference code (Keras)

Listing 1: *Imports, seed and class names*

```
# train_cnn.py  --  CNN for CIFAR-10 (reference implementation)
import os, json, time, random
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

SEED = 42
os.makedirs("results", exist_ok=True)

def set_seed(seed=SEED):
    os.environ["PYTHONHASHSEED"] = str(seed)
    random.seed(seed)
    np.random.seed(seed)
    tf.random.set_seed(seed)

CLASS_NAMES = ["airplane", "automobile", "bird", "cat", "deer",
               "dog", "frog", "horse", "ship", "truck"]
```

Listing 2: *Steps 1–4: loading, inspection, stratified split and normalization*

```
# ---- Steps 1-4: load, inspect, split, normalize ----
def load_cifar10():
    try:
        return keras.datasets.cifar10.load_data()
    except Exception:           # offline: use a local copy of the same images
        d = np.load("cifar10.npz")
        return (d["xtr"], d["ytr"]), (d["xte"], d["yte"])

(x_full, y_full), (x_test, y_test) = load_cifar10()
y_full, y_test = y_full.flatten(), y_test.flatten()
print("train", x_full.shape, x_full.dtype, "test", x_test.shape)
print("labels:", np.unique(y_full), "counts:", np.bincount(y_full))

# split by INDEX, stratified; the official test set is kept untouched
idx_train, idx_val = train_test_split(
    np.arange(len(x_full)), test_size=0.10, stratify=y_full, random_state=SEED)
assert len(set(idx_train) & set(idx_val)) == 0        # no overlap
x_train, y_train = x_full[idx_train], y_full[idx_train]
```

```
x_val,   y_val   = x_full[idx_val],   y_full[idx_val]

# statistics from the TRAINING split only
# float64 is needed here: a float32 mean over 138 million values loses precision
mean = x_train.astype("float64").mean(axis=(0, 1, 2))
std  = x_train.astype("float64").std(axis=(0, 1, 2))
def normalize(x):
    return ((x.astype("float64") - mean) / std).astype("float32")
x_train, x_val, x_test = normalize(x_train), normalize(x_val), normalize(x_test)
print("train/val/test sizes:", len(x_train), len(x_val), len(x_test))
```

Listing 3: *Steps 5–8: model, initializer, regularization, optimizer helper*

```
# ---- Steps 5-8: architecture, initializer, regularization, compile ----
def build_model(initializer="he_normal", dropout=0.5, l2=0.0):
    reg = regularizers.l2(l2) if l2 > 0 else None
    conv = dict(kernel_size=3, padding="same", activation="relu",
                kernel_initializer=initializer, kernel_regularizer=reg)
    return keras.Sequential([
        layers.Input(shape=(32, 32, 3)),
        layers.Conv2D(32, **conv),          # -> 32x32x32   896 params
        layers.MaxPooling2D(2),             # -> 16x16x32
        layers.Conv2D(64, **conv),          # -> 16x16x64   18,496 params
        layers.MaxPooling2D(2),             # -> 8x8x64
        layers.Conv2D(128, **conv),         # -> 8x8x128    73,856 params
        layers.MaxPooling2D(2),             # -> 4x4x128
        layers.Flatten(),                   # -> 2048
        layers.Dense(128, activation="relu",
                     kernel_initializer=initializer,
                     kernel_regularizer=reg), # -> 128       262,272 params
        layers.Dropout(dropout),            # -> 128       0 params
        layers.Dense(10, activation="softmax",
                     kernel_initializer="glorot_uniform"),  # 1,290 params
    ])

def make_optimizer(name, lr):
    if name == "adam":
        return keras.optimizers.Adam(learning_rate=lr)
    if name == "sgd_momentum":
        return keras.optimizers.SGD(learning_rate=lr, momentum=0.9)
    raise ValueError("unknown optimizer: " + name)

def conv_out(n, k, s, p):                   # hand-calculation helper
    return (n + 2 * p - k) // s + 1
assert conv_out(32, 3, 1, 1) == 32 and conv_out(32, 2, 2, 0) == 16

# optional augmentation, used on TRAINING images only
augment = keras.Sequential([layers.RandomFlip("horizontal"),
                            layers.RandomTranslation(0.1, 0.1)])
```

Listing 4: *Steps 9–10: training one experiment and saving the history*

```
# ---- Steps 9-10: train and save history (one experiment) ----
def run_experiment(name, optimizer="adam", lr=1e-3, initializer="he_normal",
                   dropout=0.0, l2=0.0, use_aug=False, batch=64, epochs=30):
    set_seed()
    keras.backend.clear_session()
    model = build_model(initializer, dropout, l2)
    model.compile(optimizer=make_optimizer(optimizer, lr),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    if name.startswith("A"):
        model.summary()
        n_params = model.count_params()
        assert n_params == 356810, n_params     # matches the hand calculation

    train_ds = tf.data.Dataset.from_tensor_slices((x_train, y_train))
    train_ds = train_ds.shuffle(10000, seed=SEED).batch(batch)
    if use_aug:
        train_ds = train_ds.map(lambda x, y: (augment(x, training=True), y),
```

```
                        num_parallel_calls=tf.data.AUTOTUNE)
    train_ds = train_ds.prefetch(tf.data.AUTOTUNE)

    stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=5,
                                         restore_best_weights=True)
    t0 = time.time()
    hist = model.fit(train_ds, validation_data=(x_val, y_val),
                     epochs=epochs, callbacks=[stop], verbose=2)
    seconds = time.time() - t0

    with open(f"results/{name}_history.json", "w") as f:
        json.dump({k: [float(v) for v in vs] for k, vs in hist.history.items()}, f)
    model.save(f"results/{name}.keras")
    return model, hist, seconds
```

Listing 5: *Steps 11–12: learning-curve plots*

```
# ---- Steps 11-12: learning curves ----
def plot_curves(name, hist):
    h = hist.history
    ep = range(1, len(h["loss"]) + 1)
    for key, title in [("accuracy", "Accuracy"), ("loss", "Loss")]:
        plt.figure(figsize=(6, 4))
        plt.plot(ep, h[key], label="training")
        plt.plot(ep, h["val_" + key], label="validation")
        plt.xlabel("Epoch"); plt.ylabel(title); plt.title(f"{name}: {title}")
        plt.legend(); plt.grid(alpha=0.3); plt.tight_layout()
        plt.savefig(f"results/{name}_{key}.png", dpi=150); plt.close()
```

Listing 6: *Steps 13–15: test evaluation, confusion matrix, classification report, save/load check*

```
# ---- Steps 13-15: ONE final evaluation on the test set, then record ----
def evaluate_on_test(name, model, hist, seconds, config):
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
    y_pred = np.argmax(model.predict(x_test, verbose=0), axis=1)
    cm = confusion_matrix(y_test, y_pred)
    report = classification_report(y_test, y_pred, target_names=CLASS_NAMES, digits=4)
    print(report)
    plt.figure(figsize=(7, 6))
    plt.imshow(cm, cmap="Blues"); plt.colorbar()
    plt.xticks(range(10), CLASS_NAMES, rotation=45, ha="right")
    plt.yticks(range(10), CLASS_NAMES)
    plt.xlabel("Predicted"); plt.ylabel("True"); plt.title(f"{name}: confusion matrix")
    plt.tight_layout(); plt.savefig(f"results/{name}_confusion.png", dpi=150); plt.close()
    h = hist.history
    b = int(np.argmin(h["val_loss"]))        # best epoch = the restored weights
    result = dict(config, name=name, params=int(model.count_params()),
                  epochs_run=len(h["loss"]), best_epoch=b + 1, seconds=round(seconds, 1),
                  train_acc=float(h["accuracy"][b]), val_acc=float(h["val_accuracy"][b]),
                  train_loss=float(h["loss"][b]), val_loss=float(h["val_loss"][b]),
                  test_acc=float(test_acc), test_loss=float(test_loss))
    with open(f"results/{name}_result.json", "w") as f:
        json.dump(result, f, indent=2)
    with open(f"results/{name}_report.txt", "w") as f:
        f.write(report)
    return result

# save / load check (see test T14)
def check_save_load(name, model):
    loaded = keras.models.load_model(f"results/{name}.keras")
    p1 = model.predict(x_val[:100], verbose=0)
    p2 = loaded.predict(x_val[:100], verbose=0)
    assert np.allclose(p1, p2, atol=1e-5)
```

Listing 7: *Running the experiments of Section 9*

```
# ---- Experiments A to E: see Section 9 ----
EXPERIMENTS = {
   "A_baseline":  dict(optimizer="adam", lr=1e-3, initializer="he_normal"),
   "B_regular":   dict(optimizer="adam", lr=1e-3, initializer="he_normal",
                       dropout=0.5, l2=1e-4),
   "C_sgd":       dict(optimizer="sgd_momentum", lr=1e-2, initializer="he_normal",
                       dropout=0.5, l2=1e-4),
   "D_glorot":    dict(optimizer="adam", lr=1e-3, initializer="glorot_uniform",
                       dropout=0.5, l2=1e-4),
   "E_augment":   dict(optimizer="adam", lr=1e-3, initializer="he_normal",
                       dropout=0.5, l2=1e-4, use_aug=True),
}
if __name__ == "__main__":
   all_results = []
   for name, cfg in EXPERIMENTS.items():
      model, hist, secs = run_experiment(name, **cfg)
      plot_curves(name, hist)
      check_save_load(name, model)
      # NOTE: final model choice must be made from validation results first;
      # test results are read once, after the configuration is fixed.
      all_results.append(evaluate_on_test(name, model, hist, secs, cfg))
   with open("results/all_results.json", "w") as f:
      json.dump(all_results, f, indent=2)
```

Keep one copy of the final version of the code and a requirements.txt produced with pip freeze (or a list of the library versions you used) so that another person can reproduce your results.

7.4 Environment and model summary (evidence)

Table 21: *Environment used for all runs reported in this document*

Item	Value
Python	3.13
Libraries	TensorFlow 2.21.0 (with Keras), NumPy 2.5.3, scikit-learn 1.9.1, Matplotlib 3.11.2
Hardware	Linux cloud container, 2 CPU cores, 7 GB RAM, no GPU
Random seed	42 for Python, NumPy and TensorFlow; the same seed and the same data split in every experiment
Files	train_cnn.py, tests_and_figures.py, requirements.txt, and the results/ folder (histories, models, figures)

8. Pseudocode

The following language-independent pseudocode matches the architecture of Section 5 and the code of Section 7. It describes what is done, not how any library does it.

Listing 9: *Dataset preparation*

```
ALGORITHM PrepareData
INPUT:  raw images X (N x 32 x 32 x 3), integer labels y in {0..9}, official test set (Xt, yt)
OUTPUT: normalized training, validation and test sets
BEGIN
  SPLIT indices of X into idx_train (90%) and idx_val (10%), stratified by class
  ASSERT idx_train and idx_val share no element
  X_train <- X[idx_train];  X_val <- X[idx_val]
  FOR each colour channel c in {R, G, B}
     mean[c] <- average of X_train over all images and pixels in channel c
     std[c]  <- standard deviation of X_train in channel c
  END FOR
  FOR each of X_train, X_val, Xt
     X <- (X - mean) / std         // uses TRAINING statistics only
  END FOR
  RETURN (X_train, y_train), (X_val, y_val), (Xt, yt)
END
```

Listing 10: *CNN architecture construction*

```
ALGORITHM BuildCNN
INPUT:  input shape (32, 32, 3), initializer, dropout rate p, L2 strength lambda
OUTPUT: untrained model M
BEGIN
  shape <- (32, 32, 3)
  FOR each filter count f in [32, 64, 128]
     ADD Conv2D(f filters, 3x3, stride 1, padding same, ReLU, initializer, L2 lambda)
     shape <- (height, width, f)           // size unchanged by 'same' padding
     ADD MaxPool(2x2, stride 2)
     shape <- (height / 2, width / 2, f)    // 32 -> 16 -> 8 -> 4
  END FOR
  ADD Flatten                            // 4 * 4 * 128 = 2048 values
  ADD Dense(128, ReLU, initializer, L2 lambda)
  ADD Dropout(p)
  ADD Dense(10, softmax)
  FOR each layer with parameters
     PRINT output shape and (inputs + 1) * units, or (K*K*Cin + 1) * filters
  END FOR
  ASSERT total parameters = value calculated by hand
  RETURN M
END
```

Listing 11: *Model training with early stopping*

```
ALGORITHM Train
INPUT:  model M, training data, validation data, optimizer O, learning rate eta,
        batch size B, maximum epochs E, patience Pat
OUTPUT: trained model, history
BEGIN
  best_val_loss <- infinity;  wait <- 0;  history <- empty
  FOR epoch <- 1 TO E
     SHUFFLE training data
     FOR each mini-batch (Xb, yb) of size B
        probs <- FORWARD(M, Xb)              // dropout active
        loss  <- CrossEntropy(probs, yb) + L2 penalty
        grads <- BACKPROPAGATE(loss, parameters of M)
        UPDATE parameters using optimizer O, grads and eta
     END FOR
     train_loss, train_acc <- average over training epoch
     val_loss, val_acc     <- EVALUATE(M, validation data)   // dropout off
     APPEND (train_loss, train_acc, val_loss, val_acc) to history
     IF val_loss < best_val_loss THEN
        best_val_loss <- val_loss;  SAVE weights of M;  wait <- 0
```

```
    ELSE
      wait <- wait + 1
      IF wait >= Pat THEN BREAK
    END IF
  END FOR
  RESTORE saved best weights into M
  RETURN M, history
END
```

Listing 12: *Gradient-based optimization (conceptual)*

```
ALGORITHM GradientStep     // one update; two common variants
INPUT:  parameter w, gradient g, learning rate eta, state (v, m, s, t)
OUTPUT: updated w and state
BEGIN
  IF optimizer = PLAIN_SGD THEN
    w <- w - eta * g
  ELSE IF optimizer = SGD_MOMENTUM THEN
    v <- beta * v + g
    w <- w - eta * v
  ELSE IF optimizer = ADAM THEN
    t <- t + 1
    m <- beta1 * m + (1 - beta1) * g
    s <- beta2 * s + (1 - beta2) * g * g
    m_hat <- m / (1 - beta1 ^ t);  s_hat <- s / (1 - beta2 ^ t)
    w <- w - eta * m_hat / (SQRT(s_hat) + epsilon)
  END IF
  RETURN w and state
END
```

Listing 13: *Evaluation and result reporting*

```
ALGORITHM EvaluateAndReport
INPUT:  final model M, test data (Xt, yt), class names
OUTPUT: metrics, confusion matrix, written report
BEGIN
  // called ONCE, after every design decision has been fixed using validation data
  probs <- FORWARD(M, Xt);  pred <- ARGMAX(probs per image)
  test_acc  <- fraction of images with pred = yt
  test_loss <- CrossEntropy(probs, yt)
  FOR i <- 0 TO 9 AND j <- 0 TO 9
    CM[i][j] <- number of images with true class i and predicted class j
  END FOR
  FOR each class c
    precision[c] <- CM[c][c] / (sum of column c of CM)
    recall[c]    <- CM[c][c] / (sum of row c of CM)
    f1[c]        <- 2 * precision[c] * recall[c] / (precision[c] + recall[c])
  END FOR
  RECORD configuration, parameter count, epochs run, training time and all metrics
  SAVE learning curves, confusion matrix and report
  RETURN metrics
END
```

9. Experimental Investigation

Five controlled experiments (A–E) were run with the reference CNN of Section 5, [3 marks]. Each experiment changes **one major factor** with respect to the previous reference experiment, as listed in the configuration table below. All numbers in this document come from these runs (one run per configuration, seed 42, same data split, batch size 64, at most 30 epochs with early stopping on validation loss, patience 5). Training and validation values are those of the best epoch (lowest validation loss), whose weights early stopping restores and which were then evaluated on the test set. The test accuracy is reported for every experiment so that nothing is hidden, but the final model was selected on **validation accuracy only**.

9.1 Experiment design (as run)

Table 22: *Configurations of the five experiments*

Parameter	A	B	C	D	E
Purpose	Baseline	Add regularization	Change optimizer	Change initialization	Add augmentation
Changed factor	—	Dropout 0.5 and L2 1e-4 (vs. A)	Optimizer and its learning rate (vs. B)	Initializer (vs. B)	Augmentation (vs. B)
Model	Reference design	Reference design	Reference design	Reference design	Reference design
Optimizer	Adam	Adam	SGD, momentum 0.9	Adam	Adam
Learning rate	0.001	0.001	0.01	0.001	0.001
Initialization	He normal	He normal	He normal	Glorot uniform	He normal
Dropout / L2	None / none	0.5 / 1e-4	0.5 / 1e-4	0.5 / 1e-4	0.5 / 1e-4
Augmentation	No	No	No	No	Flip + shift
Batch size	64	64	64	64	64
Epochs	Max 30, early stop (patience 5)	Max 30, early stop (patience 5)	Max 30, early stop (patience 5)	Max 30, early stop (patience 5)	Max 30, early stop (patience 5)

Notes. (1) Adam and SGD need learning rates on different scales, so Experiment C changes the optimizer and its learning rate together; the learning rates were fixed in advance from common defaults and were not tuned on any data. (2) Early stopping chooses the number of epochs, so the epochs actually run are recorded below. (3) The same seed and data split were used in every experiment.

9.2 Results record

Table 23: *Measured results of the five experiments (values at the best epoch)*

Quantity	A	B	C	D	E
Model configuration (summary)	Reference CNN	Reference CNN	Reference CNN	Reference CNN	Reference CNN
Optimizer and learning rate	Adam, 0.001	Adam, 0.001	SGD, momentum 0.9, 0.01	Adam, 0.001	Adam, 0.001
Initialization method	He normal	He normal	He normal	Glorot uniform	He normal
Regularization methods	None	Dropout 0.5, L2 1e-4	Dropout 0.5, L2 1e-4	Dropout 0.5, L2 1e-4	Dropout 0.5, L2 1e-4, flip + shift
Batch size	64	64	64	64	64
Number of epochs run (best epoch)	10 (5)	13 (8)	16 (11)	12 (7)	24 (19)
Training accuracy	79.5%	74.9%	73.9%	76.3%	75.2%
Validation accuracy	72.4%	72.4%	72.1%	74.0%	78.0%
Training loss	0.5833	0.8380	0.8392	0.7769	0.8757
Validation loss	0.8251	0.9274	0.9011	0.8506	0.7859
Train–validation accuracy gap (points)	7.1	2.5	1.8	2.3	-2.8
Test accuracy	71.2%	71.9%	72.4%	73.7%	77.7%
Test loss	0.8551	0.9518	0.9328	0.8647	0.8067
Parameter count	356,810	356,810	356,810	356,810	356,810
Training duration (seconds)	301	418	494	377	826

All five models have the same architecture, so the parameter count is the same, 356,810. Durations were measured on two CPU cores and include validation after each epoch; they vary a little with other activity on the machine.

9.3 Method followed

- Every experiment used the code of Section 7 with only the settings of the configuration table changed; the saved histories, models and result files are part of the submission.
- Experiments were compared on **validation** results; the model with the highest validation accuracy (ties: lower validation loss) was named the final model, and its test results are discussed in Section 12.
- Each configuration was run once, so differences of about one percentage point or less are within what a different seed could change and are not treated as evidence of a real effect (see the limitations in Section 12.9).

10. Learning Curves and Visualization

Learning curves, a confusion matrix and comparison charts were produced from the saved histories of all five experiments [2 marks]. In every curve plot the solid line is the training value, the dashed line is the validation value, and the dotted vertical line marks the best epoch (lowest validation loss).

10.1 Learning curves of each experiment

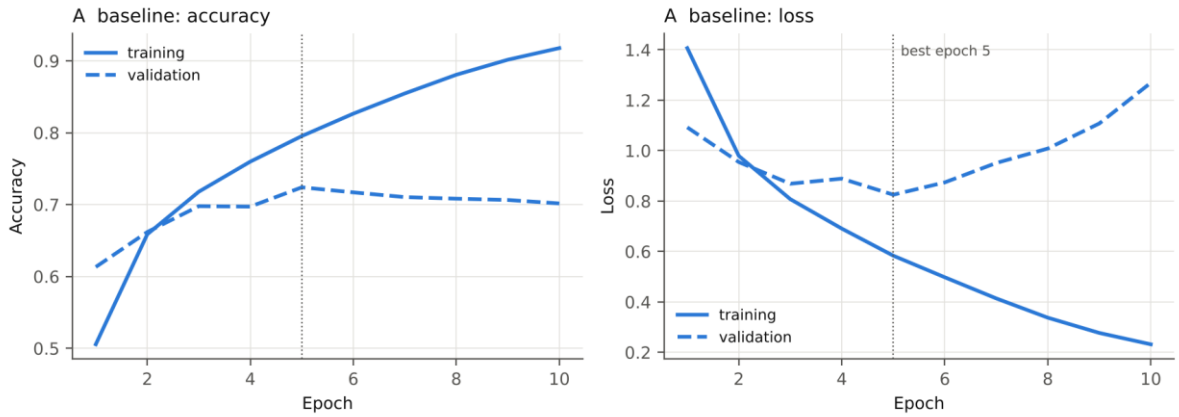


Figure 3: *Experiment A (baseline): accuracy (left) and loss (right) versus epoch*

Experiment A (baseline). Training ran for 10 epochs and stopped early; training loss fell from 1.4066 in epoch 1 to 0.2318 in epoch 10. Validation loss reached its minimum of 0.8251 at epoch 5, then rose to 1.2690 by epoch 10 while training loss kept falling, and the training–validation accuracy gap widened from 7.1 points at the best epoch to 21.6 points at the last epoch: **overfitting**.

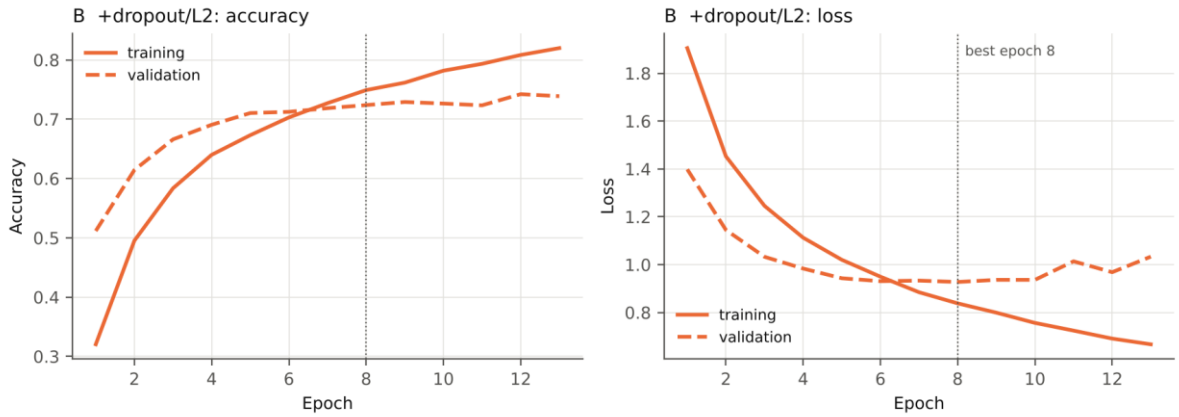


Figure 4: *Experiment B (dropout + L2): accuracy (left) and loss (right) versus epoch*

Experiment B (dropout + L2). Training ran for 13 epochs and stopped early; training loss fell from 1.9051 in epoch 1 to 0.6668 in epoch 13. Validation loss reached its minimum of 0.9274 at epoch 8, then rose to 1.0332 by epoch 13 while training loss kept falling, and the training–validation accuracy gap widened from 2.5 points at the best epoch to 8.1 points at the last epoch: **overfitting**.

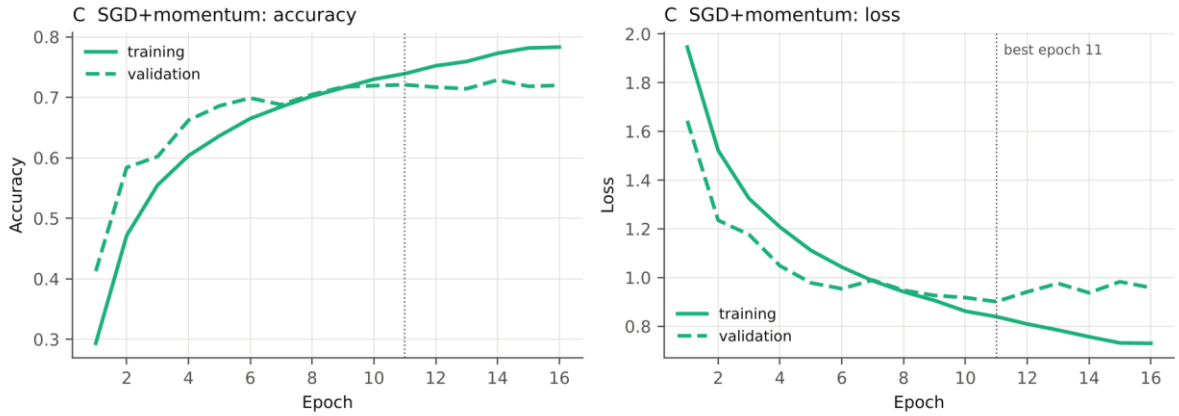


Figure 5: *Experiment C (SGD with momentum): accuracy (left) and loss (right) versus epoch*

Experiment C (SGD with momentum). Training ran for 16 epochs and stopped early; training loss fell from 1.9459 in epoch 1 to 0.7298 in epoch 16. Validation loss reached its minimum of 0.9011 at epoch 11, then rose to 0.9585 by epoch 16 while training loss kept falling, and the training–validation accuracy gap widened from 1.8 points at the best epoch to 6.3 points at the last epoch: **overfitting**.

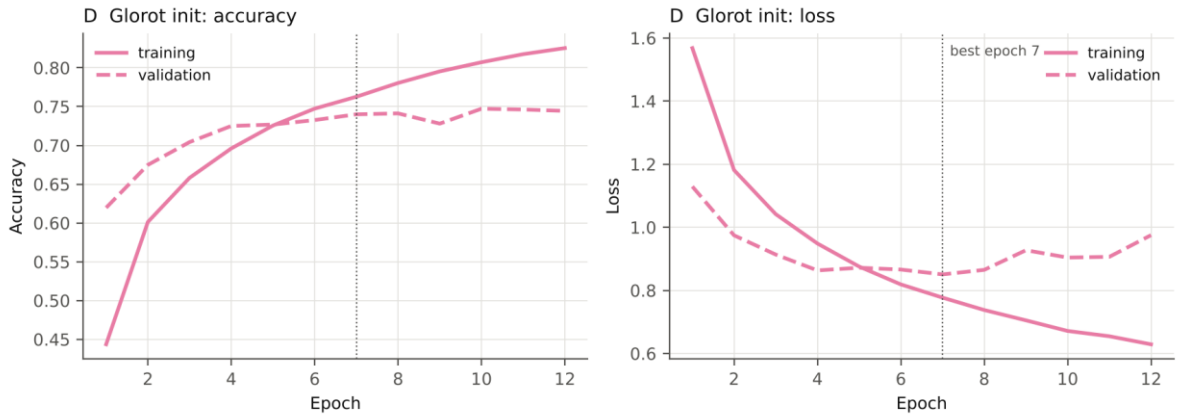


Figure 6: *Experiment D (Glorot initialization): accuracy (left) and loss (right) versus epoch*

Experiment D (Glorot initialization). Training ran for 12 epochs and stopped early; training loss fell from 1.5681 in epoch 1 to 0.6289 in epoch 12. Validation loss reached its minimum of 0.8506 at epoch 7, then rose to 0.9750 by epoch 12 while training loss kept falling, and the training–validation accuracy gap widened from 2.3 points at the best epoch to 8.1 points at the last epoch: **overfitting**.

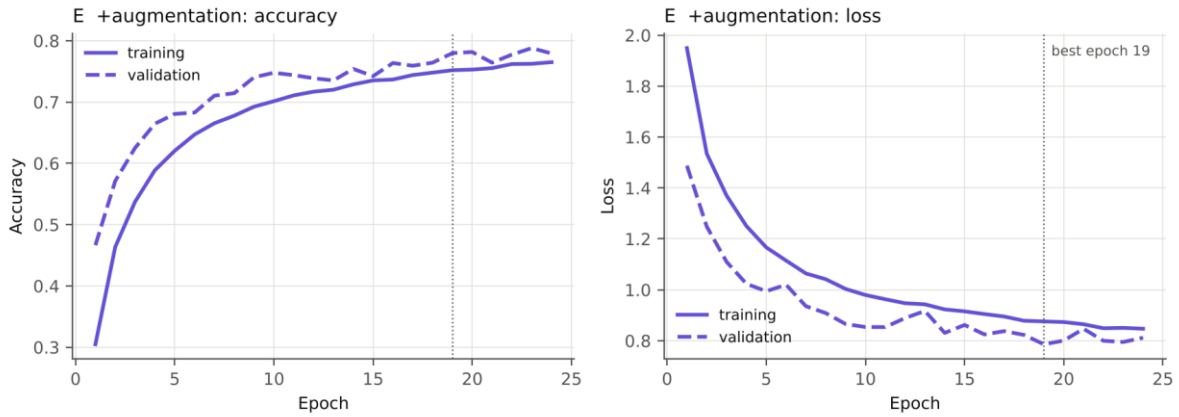


Figure 7: *Experiment E (augmentation): accuracy (left) and loss (right) versus epoch*

Experiment E (augmentation). Training ran for 24 epochs and stopped early; training loss fell from 1.9494 in epoch 1 to 0.8471 in epoch 24. Validation loss reached its minimum of 0.7859 at epoch 19; it stayed within

0.0259 of that minimum until training stopped, and the accuracy gap was -2.8 points at the best epoch and -1.4 points at the last epoch.

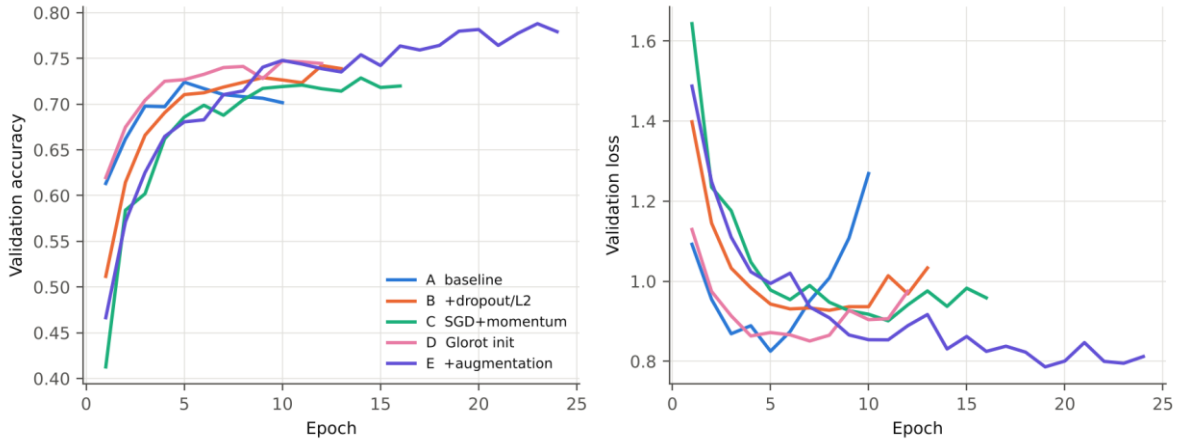


Figure 8: *Validation accuracy (left) and validation loss (right) of all five experiments*

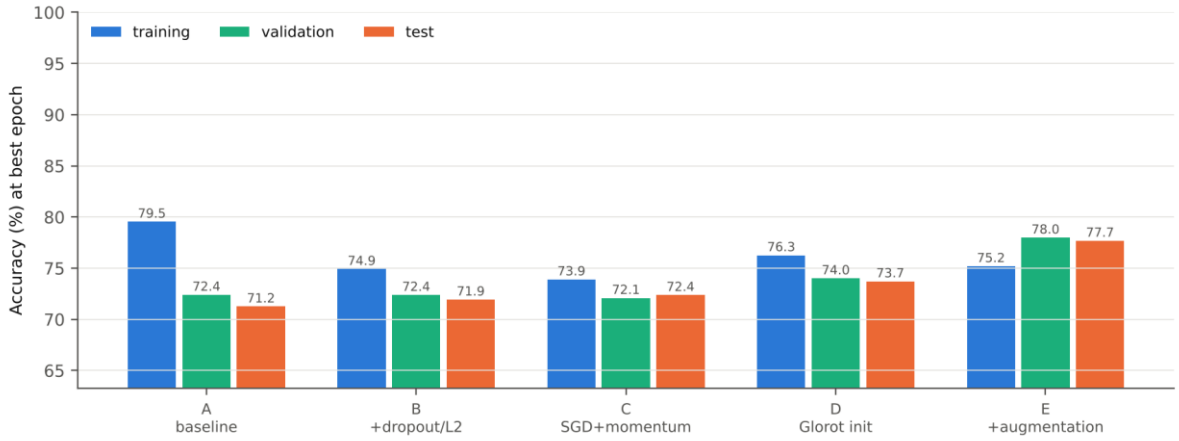


Figure 9: *Training, validation and test accuracy of the five experiments at the best epoch*

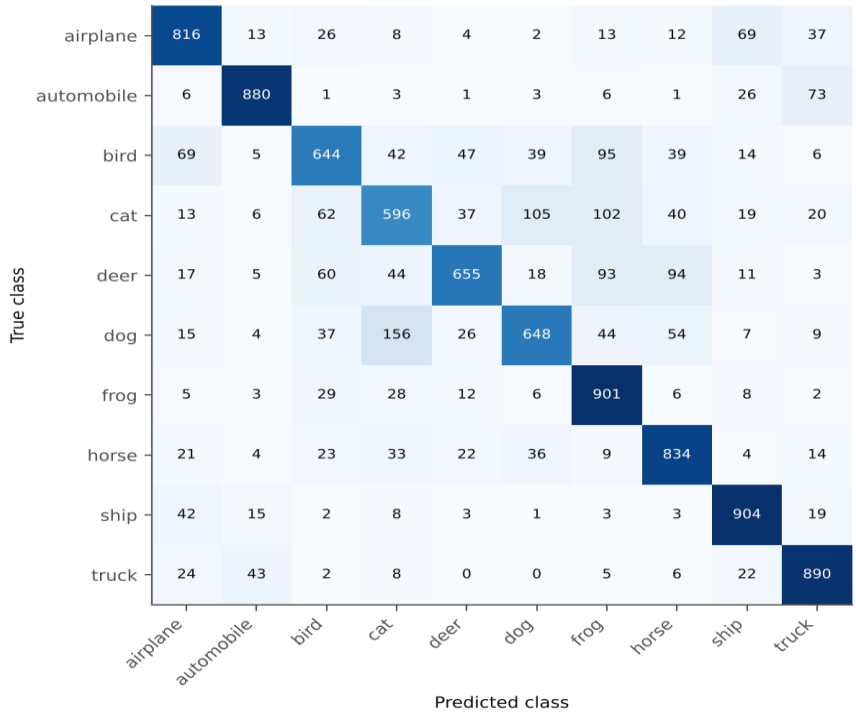


Figure 10: *Confusion matrix of the final model (Experiment E) on the 10,000 test images*

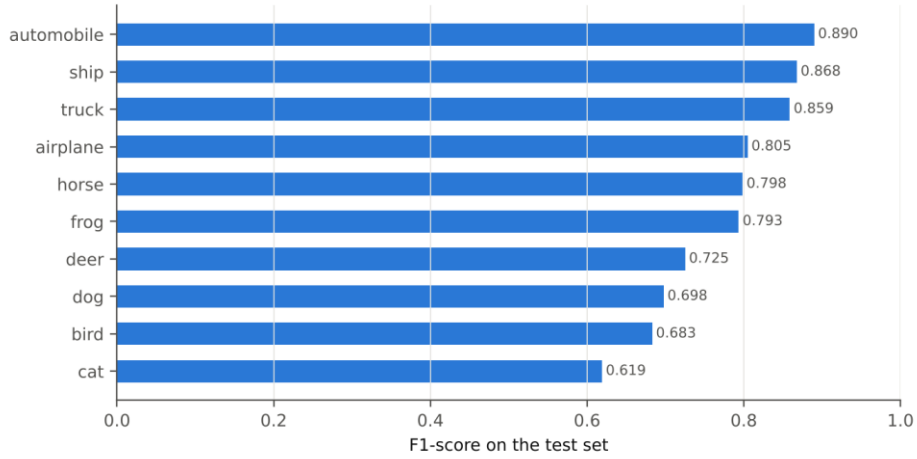


Figure 11: *F1-score of each class for the final model (Experiment E) on the test set*

10.2 How to read the curves, and what was observed

The table summarises how the usual patterns look in the curves; Section 10.1 applies it to the five experiments, quoting epochs and values from the measured histories. No pattern is judged from a single epoch.

Table 24: *Interpreting learning curves*

Pattern	What the curves look like	Likely cause	Typical actions
Convergence	Training and validation loss fall and then flatten at a similar level; validation accuracy stops improving	The model has learnt what it can with this design and these settings	Stop (early stopping); only change the design if the level is not good enough

Pattern	What the curves look like	Likely cause	Typical actions
Overfitting	Training loss keeps falling while validation loss reaches a minimum and rises; large and growing accuracy gap	Model memorizes training images; too much capacity, too little data or regularization	More dropout or weight decay, data augmentation, a smaller model, earlier stopping
Underfitting	Both losses stay high; training accuracy is low and close to validation accuracy	Model too small, too little training, learning rate too low, or poor initialization	Add filters or layers, train longer, raise the learning rate, reduce regularization
Unstable training	Loss oscillates strongly or jumps up between epochs; validation curve is erratic	Learning rate too high, noisy small batches, or bad scaling of inputs	Lower the learning rate, increase the batch size, check normalization
Learning rate too high	Loss decreases then diverges, or becomes NaN	Updates overshoot the minimum	Reduce learning rate by 3–10 times; try a learning-rate schedule
Learning rate too low	Loss decreases very slowly and steadily; far from converged after many epochs	Steps are too small	Increase the learning rate; use an adaptive optimizer

11. Testing and Validation

The 14 tests of the test plan were run on the implementation of Section 7 by tests_and_figures.py [2 marks]. The **Actual result** column quotes the values printed by that script and the **Status** column is set by the script: a test passes only if its stated condition holds. All 14 tests passed on the run reported here.

Test history (disclosed)

The **first** run of this test plan failed two tests. **T04** showed that the "normalized" training images had channel means of about 0.49, 0.46 and 0.32 and standard deviations of about 0.90, 0.91 and 0.95 instead of 0 and 1: the mean and standard deviation had been computed in float32 over 138 million values, which loses precision. They are now computed in float64 and **all five experiments were re-run**; every number in this document comes from the corrected run. (The first run gave test accuracies of 72.3%, 72.8%, 71.5%, 74.4% and 78.6% for Experiments A–E; E and D were again the two best.)

T08 failed because its first threshold, 2.0–2.6, assumed an initial loss close to ln 10. The measured initial loss is higher because He initialization gives logits with a spread well above zero (Q8, answer 2); the threshold was widened to 2.0–4.0 and the measured spread is now reported. The corrected run passes all 14 tests.

Table 25: *Test plan with actual results and status*

Test ID	Test description	Input or condition	Expected result	Actual result	Status
T01	Image dimensions and channel order	Print x_train.shape[1:] and dtype after loading	Shape (32, 32, 3); channels last; uint8 before normalization	x_train.shape[1:] = (32, 32, 3); dtype uint8; channels last	Pass
T02	Label encoding	Inspect np.unique(y) and y.shape	Integers 0–9; shape (N,); 10 classes with 5,000 training images each before splitting	labels 0-9, y.shape = (50000,), 5000 images per class	Pass
T03	Split correctness	Compare split sizes and index sets	45,000 / 5,000 / 10,000 images; train and validation indices disjoint; classes balanced in each split	sizes 45000/5000/10000; index overlap 0; 4500 train and 500 validation images per class	Pass
T04	Normalization without leakage	Mean and std computed from the training split; apply to the others	Training channel means ≈ 0 and std ≈ 1; the same constants used for validation and test	train channel means [0.0, 0.0, 0.0], stds [1.0, 1.0, 1.0]; validation means [0.011, 0.002, 0.003] (same constants applied)	Pass
T05	Output-dimension check	Compare model.summary() with your layer table	Every layer's output shape equals the hand calculation; final output (None, 10)	all 10 layer shapes equal the table; model.output_shape = (None, 10)	Pass
T06	Parameter-count verification	Compare model.count_params() and per-layer counts with your table	Equal to your calculated total (356,810 for the reference design)	per-layer [896, 0, 18496, 0, 73856, 0, 0, 262272, 0, 1290]; count_params() = 356,810	Pass
T07	Forward pass	Pass a batch of 8 normalized images through the untrained model	Output shape (8, 10); each row is non-negative and sums to 1	output shape (8, 10); row sums min 1.000000, max 1.000000; min prob 0.0001	Pass
T08	Initial loss	Evaluate the untrained model on a batch	Loss of the order of ln 10 ≈ 2.30 (a pure guess); He initialization spreads the logits, so values up to 4.0 are accepted (threshold revised, see note)	initial loss on 1,000 validation images = 3.4577 (ln 10 = 2.3026; logit spread 1.78 gives about ln 10 + spread^2/2 = 3.88)	Pass

Test ID	Test description	Input or condition	Expected result	Actual result	Status
T09	Successful training	Train for 3 epochs	No NaN or error; training loss decreases	Experiment A training loss epochs 1-3: 1.4066, 0.9792, 0.8076; no NaN in any of the 5 histories	Pass
T10	Validation monitoring	Check the history after training	Validation loss and accuracy recorded every epoch; early stopping restores the best epoch	val loss/accuracy logged every epoch; epochs run A=10 (best 5), B=13 (best 8), C=16 (best 11), D=12 (best 7), E=24 (best 19)	Pass
T11	Test-set evaluation	Evaluate the final model on the test set once	Test accuracy, confusion matrix and report produced; the test set was not used in any tuning step	x_test is used only in evaluate_on_test() (plus the one normalization line); the 5 configurations were fixed in advance and none was changed after seeing test results; final model chosen on validation accuracy (E_augment); test channel means after normalization [0.013, 0.012, 0.015]	Pass
T12	Reproducibility	Repeat one experiment with the same seed	Results the same or very close (state any GPU non-determinism)	two fresh runs, same seed: val loss [1.0923, 0.9546] vs [1.0923, 0.9546] (max difference 0.0000); difference to the original Experiment A run 0.0000	Pass
T13	Error handling	Give an image of the wrong shape, e.g. (28, 28, 3), or an unknown optimizer name	A clear error is raised, not a silent wrong result	wrong-shape input raised InvalidArgumentError; unknown optimizer name raised ValueError	Pass
T14	Saving and loading	Save the model, reload it and predict on 100 validation images	Predictions of the original and reloaded model agree (np.allclose)	max |difference| between original and reloaded predictions = 0.00e+00	Pass

12. Results and Discussion

This section reports and discusses the measured results [3 marks]. All values come from the runs of Section 9 (one run per configuration, seed 42). The test accuracy of every experiment is shown, but the final model, **Experiment E**, was chosen because it had the highest **validation** accuracy (78.0%, against 74.0% for the next best, D).

12.1 Summary of final results

Item	Value (final model, Experiment E)
Chosen final configuration and why	Reference CNN, He-normal initialization, Adam (0.001), dropout 0.5, L2 1e-4, flip and shift augmentation; highest validation accuracy of the five experiments
Parameter count	356,810 (all trainable)
Epochs run / training time	24 epochs (best epoch 19) / 826 s on two CPU cores
Final training accuracy and loss (best epoch)	75.2% / 0.8757
Final validation accuracy and loss (best epoch)	78.0% / 0.7859
Test accuracy and test loss	77.68% / 0.8067
Macro-average precision, recall, F1 (test)	0.7769 / 0.7768 / 0.7737

The losses of Experiments B–E include the L2 penalty $\lambda\Sigma w^2$ and are therefore larger than the pure cross-entropy; accuracies are used for comparisons across experiments.

12.2 Final test performance

The final model classifies **77.68%** of the 10,000 test images correctly (test loss 0.8067). The validation accuracy at the same epoch was 77.98%, only 0.30 points higher, so there is no sign that the choice of the best epoch on the validation set made the test estimate optimistic. The per-class results of the test set are given below.

Table 26: *Classification report of the final model on the test set*

Class	Precision	Recall	F1-score	Support
airplane	0.7938	0.8160	0.8047	1000
automobile	0.8998	0.8800	0.8898	1000
bird	0.7269	0.6440	0.6829	1000
cat	0.6436	0.5960	0.6189	1000
deer	0.8116	0.6550	0.7250	1000
dog	0.7552	0.6480	0.6975	1000
frog	0.7089	0.9010	0.7935	1000
horse	0.7658	0.8340	0.7985	1000
ship	0.8339	0.9040	0.8676	1000

Class	Precision	Recall	F1-score	Support
truck	0.8295	0.8900	0.8587	1000
Macro average	0.7769	0.7768	0.7737	10000
Accuracy			**0.7768**	**10000**

The best classes are **automobile** (F1 0.8898), **ship** (0.8676) and **truck** (0.8587): vehicles with distinctive shapes and backgrounds. The worst are **cat** (0.6189), **bird** (0.6829) and **dog** (0.6975): animals that vary widely in pose and look alike. The most frequent confusions in the confusion matrix are dog predicted as cat (156 images), cat as dog (105), cat as frog (102), bird as frog (95) and deer as horse (94). These pairs share fur or texture, body shape and green or brown backgrounds, and at 32 × 32 pixels fine detail such as ears and snouts is almost lost. Deer have the highest precision among the animals (0.8116) but a recall of only 0.6550: the network is cautious about predicting deer, and many deer are called frog or horse.

12.3 Learning curves

For the final model (Figure for Experiment E in Section 10) training loss fell from 1.9494 in epoch 1 to 0.8471 in epoch 24, and validation loss reached its lowest value, 0.7859, at epoch 19; it was 0.8118 at the last epoch, only 0.026 above the minimum, when the patience of 5 epochs ran out. Validation accuracy rose from 46.6% in epoch 1 to 78.0% at the best epoch and stayed within about one point of it afterwards. Validation was **above** training accuracy during the whole run (76.5% against 77.9% in the last epoch), because the training accuracy is measured on augmented images with dropout active, whereas validation uses the clean images with dropout off. The curves show slow, steady convergence without divergence or oscillation.

12.4 Parameter counts

The hand-calculated count of 356,810 equals the count reported by Keras (test T06 and the table in Section 5.3). The first dense layer holds 262,272 parameters (73.5%), the three convolutional layers together 93,248 (26.1%) and the output layer 1,290 (0.4%). This is what was expected from the calculation of Section 6: the dense layer dominates because Flatten feeds 2,048 values into 128 units, and so dropout and weight decay were placed on the dense part.

12.5 Effect of initialization

Experiment D repeated Experiment B with Glorot-uniform instead of He-normal initialization. Glorot gave a validation accuracy of 74.0% against 72.4% for He (+1.6 points) and a test accuracy of 73.7% against 71.9% (+1.8 points), and its best epoch was earlier (7 against 8). The theory of Q5 would have led to the expectation that He is at least as good for ReLU; the measurement does not support an advantage for He in this network. Two reasons make a firm conclusion impossible: each result comes from a single random draw of the initial weights, and a difference of about 1.6 points is of the size that a change of seed can produce; and a network with only three convolutional layers is shallow, so the signal does not shrink or grow enough over its depth for the two schemes to differ strongly. **No claim is made that either initializer is generally better.**

12.6 Effect of optimizer choice

Experiments B (Adam) and C (SGD with momentum) differ in optimizer and learning rate. Adam learned faster at first: after epoch 3 its validation accuracy was 66.6% against 60.2%. SGD needed 16 epochs (best epoch 11) against 13 (best epoch 8), but reached practically the same validation accuracy (72.1%

against 72.4%) and test accuracy (72.4% against 71.9%); its train–validation gap at the best epoch was the smallest of all runs (1.8 points). Both trained stably. The learning rates were not tuned, and the learning rate changed together with the optimizer, so the comparison shows the speed difference but cannot rank the optimizers in final quality.

12.7 Effect of regularization

Dropout and L2 (A → B). The gap between training and validation accuracy at the best epoch fell from 7.1 to 2.5 points, and at the end of training from 21.6 points (epoch 10) to 8.1 points (epoch 13). Training accuracy at the best epoch dropped from 79.5% to 74.9%, while validation accuracy was unchanged (72.4% against 72.4%); test accuracy was 0.7 points higher. Regularization therefore controlled overfitting but did not by itself improve accuracy.

Augmentation (B → E). Adding flips and shifts raised validation accuracy by 5.6 points and test accuracy by 5.8 points and removed the overfitting pattern (Section 12.8). It was by far the largest effect measured and the only one clearly larger than the run-to-run differences expected from a change of seed. It cost 2 times the training time of B.

12.8 Evidence of overfitting or underfitting

- **Experiment A shows clear overfitting.** Training accuracy rose to 91.8% by epoch 10 while validation accuracy stayed near 72.4%; validation loss was lowest at epoch 5 (0.8251) and rose to 1.2690 by epoch 10, a gap of 21.6 points.
- **Experiment B shows weaker, later overfitting.** Its validation loss was lowest at epoch 8 (0.9274) and rose to 1.0332 by epoch 13.
- **Experiments C and D** overfit in the same way as B, at epochs 11 and 7.
- **Experiment E shows neither strong overfitting nor strong underfitting.** Validation loss stayed within 0.026 of its minimum and validation accuracy was above training accuracy. The model is limited more by its capacity and training time (training accuracy only 76.5%) than by memorization, which suggests that further gains would come from a stronger model or longer training, not from more regularization.

12.9 Limitations of the model and the study

- **One run per configuration.** Differences of about one point or less (for example A against B against C, and the 1.6 points between B and D) are within what a different seed could change. Repeated runs with several seeds, with mean and range, were not done.
- **Little tuning.** The learning rates, dropout rate, λ and batch size were fixed in advance from common defaults. A better tuned optimizer or regularization strength could change the ranking of B, C and D.
- **Dataset copy.** The images came from a public GitHub mirror of CIFAR-10 because the official site was not reachable; its class counts and image sizes match the specification, but it was not compared byte-for-byte with the official files (Section 2.2).
- **Capacity and resolution.** The network is small (356,810 parameters) and the images are only 32 × 32 pixels; 77.7% accuracy is well below what deeper networks with batch normalization achieve on this data set.
- **Loss comparison.** The losses of B–E include the L2 penalty, so only accuracies and within-experiment curves are compared.

- **Test set use.** The test accuracy of all five experiments is reported, but only validation results were used to choose the final model; no setting was changed after seeing a test result.
- **Generalization.** The conclusions apply to this network and data set; they should not be taken as general rules about initializers or optimizers.

12.10 Possible improvements

- **Batch normalization and a deeper network** (for example two convolutions per block): expected to speed up and stabilize training and raise accuracy. Test by adding the layers to Experiment E and comparing validation accuracy over several seeds.
- **Learning-rate schedule** (reduce on plateau or cosine decay): Experiment E stopped by patience while still improving slowly. Test by comparing validation accuracy with and without the schedule.
- **Stronger augmentation and longer training** (random crops, cutout, 60+ epochs): augmentation was the most effective technique, so more of it may help. Test with the same architecture and a larger epoch budget, comparing validation accuracy and the train–validation gap.
- **Several seeds and an ensemble of models:** averages the predictions of independently trained networks and also quantifies the run-to-run variation that Section 12.9 could not measure.
- **Global average pooling instead of Flatten:** would reduce the first dense layer from 262,272 to 16,512 parameters and so reduce overfitting; test by comparing parameter counts, validation accuracy and gap.

13. Marking Scheme

Proposed rubric
No official marks distribution was given in the supplied assignment description. The 50-mark scheme below is a **proposal** for faculty to accept or change. It adds up to exactly 50.

Table 27: *Proposed allocation of the 50 marks*

Component	Marks
C1. Problem understanding and dataset preparation	5
C2. CNN architecture design and output-shape calculations	8
C3. Trainable parameter calculations	7
C4. Initialization, optimizer, and gradient-based training	8
C5. Regularization and experimental design	7
C6. Implementation, testing, and validation	5
C7. Learning curves, test results, and discussion	7
C8. Report quality and reproducibility	3
Total	**50**

13.1 What earns full, partial and minimal marks

Table 28: *Marking criteria (proposed)*

Component (marks)	Full marks	Partial marks	Minimal marks
C1. Problem understanding and dataset preparation (5)	The problem, classes, input format and splits are stated correctly and in your own words; the dataset is loaded, inspected, split without overlap, and normalized with training-only statistics; official requirements and assumptions are distinguished.	Most items present, but one is missing or wrong (for example, normalization statistics computed on all data without comment); little justification.	The dataset is loaded but the split, normalization or problem statement is missing or seriously wrong.
C2. CNN architecture design and output-shape calculations (8)	All layer output shapes in Q1–Q3 and the architecture table are correct with the working shown; the design contains all required elements and each choice is justified; the table agrees with the framework summary.	Method understood but with arithmetic slips, an incomplete table, or too little justification.	Formulas misapplied or shapes mostly missing or wrong; no check against the framework.
C3. Trainable parameter calculations (7)	Every trainable-parameter count (Q4 and the table) is correct with the formula and substitution shown; biases and non-trainable quantities are handled correctly; the total agrees with count_params().	Correct method, but one or two layers wrong or working missing; total reconciled only after errors.	Parameter counts mostly missing or wrong; formula not used.
C4. Initialization, optimizer, and gradient-based training (8)	Initialization and optimizer are chosen and justified from theory and evidence; the gradient-descent and loss exercises are correct with full working; the training procedure is correct and explained.	Choices stated with weak justification, or exercises partly wrong; training works but is not explained.	No justification; exercises mostly missing; training procedure incorrect.

Component (marks)	Full marks	Partial marks	Minimal marks
C5. Regularization and experimental design (7)	Suitable regularization is applied and its mechanism explained; at least three controlled experiments change one factor at a time; all results are measured and recorded in the tables; the comparison is honest about limitations.	Regularization applied but experiments are not properly controlled, or tables incomplete; comparison superficial.	No regularization or fewer than two comparable experiments; values missing or unverifiable.
C6. Implementation, testing, and validation (5)	The code runs end-to-end, covers all 15 steps, keeps the test set out of tuning, and the test plan is executed with actual results and status filled in.	Code runs but misses some steps, or tests are partly run or filled in.	Code incomplete or does not run; test plan not executed.
C7. Learning curves, test results, and discussion (7)	Learning curves for each experiment, confusion matrix and classification report are real, labelled and correctly interpreted; the discussion covers every prompt in Section 12 with evidence and limitations.	Plots present but interpretation shallow or some prompts skipped.	Missing or unlabelled plots; results unsupported or not discussed.
C8. Report quality and reproducibility (3)	Report is complete, well organized and clearly written; all deliverables are present; seeds, versions and settings are recorded so the results can be reproduced.	Report adequate but some deliverables or reproducibility details missing.	Disorganized report; key deliverables or reproducibility information missing.

Typical bands: full = 90–100% of the component, partial = 40–89%, minimal = below 40%. The marker may award any value within a band.

13.2 Where each mark comes from

The table maps every graded question and task in this document to a rubric component. The marks of the questions and tasks in each component add up to the component total.

Table 29: *Mapping of questions and tasks to the 50 marks*

Item	Description	Component	Marks
Task 1	Problem statement write-up and dataset preparation (Sections 2 and 7, steps 1–4)	C1	5
Q1	CNN architecture and image classification	C2	1
Q2	Convolution output shape	C2	2
Q3	Pooling operations	C2	1
Task 2	Architecture design task: layer table with output shapes (Section 5)	C2	4
Q4	Parameter count	C3	3
Task 3	Architecture design task: trainable-parameter column and total (Section 5)	C3	4
Q5	Weight initialization	C4	2
Q6	Gradient-based optimization	C4	3
Task 4	Practice exercises on gradient updates and loss (Section 6.9)	C4	3
Q7	Regularization	C5	2
Q10	Experimental analysis	C5	2
Task 5	Controlled experiments and result tables (Section 9)	C5	3
Task 6	Python implementation (Sections 7 and 8)	C6	3
Task 7	Test plan execution (Section 11)	C6	2
Q8	Loss functions and evaluation metrics	C7	1

Item	Description	Component	Marks
Q9	Overfitting and underfitting	C7	1
Task 8	Learning curves and visualization (Section 10)	C7	2
Task 9	Results and discussion (Section 12)	C7	3
Task 10	Report quality, deliverables and reproducibility (Section 14)	C8	3
	Total		**50**

14. Deliverables

Submission quality and completeness are assessed under [3 marks].

14.1 Required deliverables

No.	Deliverable	Format	Status and location
1	Completed Word report	.docx	Complete: this document (the student's personal details are entered on the cover page)
2	CNN architecture diagram or table	Figure and table in the report	Complete: Figure 2 and Sections 5.2–5.3
3	Output-shape and parameter calculations	In the report	Complete: Sections 4 (Q2–Q4), 5 and 6
4	Source code files	.py	Complete: train_cnn.py and tests_and_figures.py (Section 7 and the submitted folder)
5	Requirements or environment information	requirements.txt and table	Complete: Section 7.4 and requirements.txt
6	Experiment configuration and results	Tables and saved result files	Complete: Section 9 and results/*_result.json
7	Learning-curve graphs	Images in the report	Complete: Section 10, one figure per experiment plus overlays
8	Confusion matrix and classification report	Image and table in the report	Complete: Sections 10 and 12 (final model, test set)
9	Test evidence	Completed test table	Complete: Section 11, 14 tests run
10	Brief conclusion and future improvements	Section 16	Complete: written from the measured results

14.2 Optional supporting materials

- The two optional experiments (D and E) were run in addition to the three required ones.
- A drawn architecture diagram (Figure 2) and a comparison chart across experiments (Section 10).
- The saved model files (.keras) and the training histories are included in the results/ folder of the submitted files.
- Not done: repeated runs with several seeds and a PyTorch version (see the limitations in Section 12.9).

16. Conclusion

In this assignment a CNN with three convolution–pooling blocks and a dense classifier (356,810 trainable parameters) was designed and trained on CIFAR-10 images. The output shape and parameter count of every layer were calculated by hand before training, using $\lfloor (n + 2P - K)/S \rfloor + 1$ and $(K^2C + 1)F$, and agreed exactly with the framework summary. The network was trained with He-normal initialization and the Adam optimizer, and five controlled experiments compared a baseline with dropout and L2 regularization, SGD with momentum, Glorot initialization and data augmentation.

The final model (Experiment E, with dropout, L2 and flip-and-shift augmentation) reached a validation accuracy of 78.0% and a **test accuracy of 77.68%** on the 10,000 unseen test images (macro F1 0.7737). Without augmentation the models reached 71.2%–73.7% test accuracy. The baseline overfitted clearly (training accuracy 79.5% against 72.4% validation at its best epoch, validation loss rising after epoch 5); dropout and L2 narrowed the gap from 7.1 to 2.5 points without raising validation accuracy; SGD with momentum learned more slowly but ended at a similar accuracy; Glorot initialization was not worse than He in this shallow network; and augmentation was the most effective measure (+5.6 points validation accuracy over B). Cats, birds and dogs were the hardest classes and automobiles, ships and trucks the easiest.

These conclusions are limited by one run per configuration, by untuned learning rates and regularization strengths, by the small network and the 32 × 32 image size, and by the use of a public mirror of the data set. Future work could add batch normalization and more depth, a learning-rate schedule, stronger augmentation with longer training, and repeated runs with several seeds, as set out in Section 12.10. The sources of the formulas and methods are listed in Section 17.

17. References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. https://www.deeplearningbook.org
2. Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images* (Technical report). University of Toronto. CIFAR-10 and CIFAR-100 datasets: https://www.cs.toronto.edu/~kriz/cifar.html
3. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
4. Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 9, 249–256.
5. He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 1026–1034.
6. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*. arXiv:1412.6980.
7. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.
8. Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
9. Sutskever, I., Martens, J., Dahl, G., & Hinton, G. (2013). On the importance of initialization and momentum in deep learning. *Proceedings of the 30th International Conference on Machine Learning (ICML)*, PMLR 28(3), 1139–1147.
10. Keras documentation. Layers, optimizers, initializers, regularizers and callbacks API. https://keras.io/api/
11. TensorFlow documentation. https://www.tensorflow.org/api_docs
12. PyTorch documentation. https://pytorch.org/docs/stable/index.html
13. scikit-learn documentation. Model evaluation: confusion_matrix and classification_report. https://scikit-learn.org/stable/modules/model_evaluation.html